

RetailSync: Super Shop Warehouse Management System

Comprehensive Engineering Specification Suite & Technical Blueprint

ACADEMIC PROJECT METADATA

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)
Student / Author: Raisul Islam Likhon (Section: SWE-44D)
Institution: Daffodil International University (DIU), Dhaka, Bangladesh
Submission Term: Fall 2026 | Version: 1.0.0-RELEASE (Master Production Specification)

Specification Modules Included in this Volume

Module #	Document Name	Primary Technical Focus	Target Output Format
Module 1	Product Requirements Document (PRD)	Personas, User Journeys, Epics, Quantitative SLOs	PRD.md / PRD.docx
Module 2	Software Architecture Document (SAD)	4-Tier Architecture, Concurrency, FEFO Data Flows	ARCHITECTURE.md / ARCHITECTURE.docx
Module 3	Technology Stack & ADRs	Next.js 14, FastAPI, PostgreSQL 16, Redis 7, Hardware	TECH_STACK.md / TECH_STACK.docx
Module 4	Database Schema & Data Dictionary	3NF Relational Model, Triggers, Indexes, DDL Tables	DATABASE_SCHEMA.md / DATABASE_SCHEMA.docx
Module 5	REST API Specification & Contract	OpenAPI 3.1 Endpoints, Request/Response Envelopes	API_SPECIFICATION.md / API_SPECIFICATION.docx
Module 6	Sprint Implementation Plan & Tasks	14-Week Agile Roadmap, Story Points, DoD Criteria	SPRINT_PLAN_AND_TASKS.md / SPRINT_PLAN_AND_TASKS.docx

MODULE 1: PRODUCT REQUIREMENTS DOCUMENT (PRD)

Product Requirements Document (PRD)

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Real-Time POS Synchronization, Dynamic Putaway, FEFO Batch Tracking, and Algorithmic Replenishment

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Executive Summary & Product Vision

1.1 The Vision

RetailSync is a high-performance, centralized enterprise Warehouse Management System (WMS) purpose-built for the fast-evolving modern grocery retail and super shop sector in Bangladesh (e.g., Shwapno, Agora, Meena Bazar, Unimart, Daily Shopping).

RetailSync bridges the destructive chasm between **central distribution centers** and **frontline retail checkout counters (POS)**. By enforcing strict Third Normal Form (3NF) relational database integrity, sub-2.0s transactional POS inventory deductions, dynamic spatial putaway, and automated First-Expired, First-Out (FEFO) batch allocation, RetailSync eliminates the three largest profit leaks in modern trade:

1. **Perishable food & dairy spoilage** (15% to 22% annual loss).
2. **Phantom inventory & unrecorded shrinkage** (1.8% to 2.4% inventory write-offs).
3. **High-velocity stockouts during festival rush periods** (7.5% to 11.2% lost revenue).

1.2 Core Product Value Propositions

- **Sub-2.0 Second POS Concurrency Sync:** Cashiers process high-speed barcode sales while the backend executes atomic row-level locks (**SELECT ... FOR UPDATE**), preventing overselling and phantom stock.
 - **Strict FEFO (First-Expired, First-Out) Priority Queues:** Eliminates human picking bias by dynamically routing pickers to the batch with the nearest expiration date, backed by automated quarantine locks.
 - **Algorithmic Replenishment Decision Support System (DSS):** Directly integrates Dynamic Economic Order Quantity (EOQ), Greasley's Statistical Safety Stock (factoring demand and vendor lead-time variances), and dynamic Reorder Points (ROP) to replace managerial guesswork with mathematical precision.
 - **Frugal & Ergonomic Hardware Strategy:** Built as a responsive Progressive Web Application (PWA) supporting consumer Android smartphones paired with low-cost Bluetooth barcode trigger grips (< 4,000 BDT) or built-in camera scanning, eliminating the need for expensive 60,000 BDT ruggedized industrial terminals.
 - **Hybrid Enterprise Edge Extensibility:** Native architectural compatibility with stationary automated dock gates (ESP32 MCU + RFID / fixed scanners over MQTT/TLS) for high-throughput distribution center intake.
-

2. Target Personas & User Journeys

RetailSync explicitly addresses the real-world operational challenges of six core user personas:

2.1 Persona 1: Cashier (Frontline Retail POS)

- **Demographics:** Frontline staff, fast-paced environment, high customer pressure during rush hours (Ramadan, Eid, weekend evenings).
- **Primary Objective:** Scan items rapidly, execute sales transactions in under 2 seconds, and never experience system freezes.
- **Key Frustrations:** System freezing when checking warehouse stock; customer disputes over out-of-stock items; having to restart sales due to internet drops.
- **RetailSync User Journey:**
 4. Cashier scans barcode on consumer item.
 5. POS client interrogates RetailSync local/central deduction endpoint.
 6. RetailSync locks the earliest active FEFO batch in the database, decrements inventory by the scanned quantity, and returns confirmation in **< 350 ms**.
 7. If internet disconnects, POS writes to an offline IndexedDB queue, displays green indicator, and bulk-syncs chronologically when connection restores.

2.2 Persona 2: Inbound Receiving Clerk (Warehouse Dock)

- **Demographics:** Operates on concrete loading docks; inspects supplier delivery trucks; exposed to dust and noise.
- **Primary Objective:** Verify delivered goods against Purchase Orders (POs) quickly, reject damaged goods, and generate digital Goods Receipt Notes (GRNs).
- **Key Frustrations:** Paper manifests with missing pages; manual math errors on carton counts; being blamed for accepting near-expiry dairy batches.
- **RetailSync User Journey:**
 8. Clerk selects open PO from tablet/phone.
 9. Scans incoming master carton barcodes; inputs batch lot number and manufacturer expiry date.
 10. System automatically evaluates minimum acceptable shelf-life threshold (e.g., rejects batch if **< 75%** remaining shelf life).
 11. Clerk flags 2 crushed cartons as "Damaged/Quarantine"; completes GRN. System prints internal pallet/carton barcode labels with internal tracking IDs.

2.3 Persona 3: Warehouse Floor Operator (Putaway & Picking)

- **Demographics:** Moves constantly across warehouse aisles; operates forklifts or manual pallet jacks; one hand often occupied.
- **Primary Objective:** Know exactly where to place incoming pallets and which bins to pick items from with zero wasted walking distance.
- **Key Frustrations:** Wandering through warehouse searching for empty bins; misplaced boxes causing delayed store shipments; hard-to-read mobile screens in dim racking aisles.
- **RetailSync User Journey:**
 12. Operator scans internal pallet barcode on receiving dock.
 13. RetailSync displays Directed Putaway prompt: **Zone: Chilled Dairy -> Aisle A02 -> Rack R04 -> Shelf S1 -> Bin B03**.
 14. Operator travels to bin, scans destination bin barcode to confirm physical placement. Database updates location atomically.

15. For outbound orders, operator receives digital pick list sorted by shortest aisle path, scanning each bin and item barcode to confirm FEFO batch compliance.

2.4 Persona 4: Inventory Floor Supervisor

- **Demographics:** Tactical floor manager; responsible for stock accuracy, shift productivity, and loss prevention.
 - **Primary Objective:** Maintain > 99.5% bin location accuracy, conduct non-disruptive cycle counts, and investigate shrinkage.
 - **Key Frustrations:** Painful annual physical stock counts; unexplained missing stock discovered days after theft/breakage; unrecorded stock adjustments.
 - **RetailSync User Journey:**
16. Supervisor monitors real-time 2D spatial bin occupancy map on tablet.
 17. Schedules automated blind cycle counts for Class-A velocity SKUs (expected quantities hidden from counters).
 18. Reviews variance reports; approves adjustments with mandatory photographic proof and reason codes (Breakage, Theft, Spoilage).
 19. Reviews Isolation Forest ML anomaly alerts flagging abnormal shrinkage spikes in specific zones or shifts.

2.5 Persona 5: Procurement & SCM Officer

- **Demographics:** Office-based supply chain strategist; manages millions of BDT in purchasing budgets.
 - **Primary Objective:** Prevent stockouts of high-velocity goods while minimizing capital tied up in slow-moving inventory.
 - **Key Frustrations:** Overstocking items that expire on shelves; supplier delivery delays causing sudden stockouts; calculating order sizes using intuition.
 - **RetailSync User Journey:**
20. Officer opens Procurement Queue dashboard.
 21. System highlights SKUs breaching dynamic Reorder Point ($ROP = (d * L) + SS$).
 22. System provides pre-calculated Economic Order Quantity (EOQ) and Greasley Safety Stock based on historical lead-time standard deviation.
 23. Officer reviews suggested order with one click, modifies quantity if promotional demand is expected, and issues digital PO to supplier.

2.6 Persona 6: Branch Super Shop Store Manager

- **Demographics:** Manages a retail outlet (e.g., Shwapno Dhanmondi); responsible for branch revenue, shelf fullness, and customer satisfaction.
 - **Primary Objective:** Ensure retail store shelves are never empty and received consignments match what was ordered.
 - **Key Frustrations:** Calling central warehouse repeatedly to check dispatch status; receiving items expiring in 3 days; receiving wrong quantities.
 - **RetailSync User Journey:**
24. Manager logs into Branch Requisition Portal; views live central warehouse stock availability.
 25. Submits weekly store stock replenishment order.
 26. Tracks dispatch status in real time: Submitted -> Pick Wave Allocated -> In-Transit (Driver: Jamal, Truck: DHK-METRO-11-2345) -> Delivered.
 27. Scans received delivery totes; confirms receipt and closes requisition.

3. Scope Boundaries & MoSCoW Prioritization

3.1 In-Scope: Capstone MVP (Must Have - Sprints 1 to 4)

- **User Authentication & RBAC:** Secure JWT authentication with 6 distinct roles (Cashier, Clerk, Operator, Supervisor, Procurement, Admin).
- **Master Catalog Management:** Hierarchical product classification (Dry FMCG, Perishable Produce, Chilled Dairy, Frozen, Household) with barcode, dimensions, and temperature attributes.
- **Supplier & Purchase Order Lifecycle:** Digital PO generation, multi-tier status lifecycle (**DRAFT**, **ISSUED**, **PARTIAL**, **COMPLETED**, **CANCELLED**).
- **Digital Inbound Receiving & GRN:** Handheld barcode scanning against PO line items, variance capture, damaged quarantine tagging, and digital GRN generation.
- **Directed Spatial Putaway:** Zone-Aisle-Rack-Shelf-Bin 2D topology with volumetric and weight capacity constraints.
- **Real-Time FEFO/FIFO Inventory Ledger:** Immutable double-entry transaction ledger; automated allocation of earliest-expiring batch; quarantine lock on expired/damaged lots.
- **Sub-2.0s POS Concurrency Deduction:** High-performance REST API supporting atomic stock deductions with row-level locking (**SELECT ... FOR UPDATE**).
- **Dynamic Decision Support System (DSS):** Mathematical formulas for EOQ, Greasley's Statistical Safety Stock ($Z = 1.28, 1.65, 2.33$), and dynamic ROP alerting.

3.2 Should Have: Enhancements (Sprints 5 to 6)

- **Outbound Wave & Batch Picking:** Consolidated picking across multi-branch requisitions with shortest-path aisle sequencing.
- **Store Requisition & Dispatch Portal:** Branch-specific ordering, in-transit dispatch manifest generation, and delivery acknowledgement.
- **Continuous Blind Cycle Counting:** ABC-classified stocktaking schedules with blind quantity entry and supervisor adjustment workflows.
- **Offline POS Caching & Auto-Sync:** Service Worker and IndexedDB client-side cache allowing cashiers to continue scanning during internet dropouts.
- **Supplier SLA Performance Scorecard:** Automated metrics on vendor on-time delivery rate, fulfillment accuracy, and lead-time standard deviation.

3.3 Could Have: Future Enterprise Features (Post-Capstone)

- **Unsupervised ML Anomaly Detection:** Isolation Forest algorithm detecting unusual shrinkage clusters across SKUs, operators, and shifts.
- **Automated Dock Gate IoT Portal:** Integration with stationary ESP32 + RC522 RFID / fixed barcode readers over TLS-encrypted MQTT.
- **IoT Cold-Chain Telemetry:** Real-time BLE temperature sensor logging attached to refrigerated transport totes.
- **Customer-Facing Self-Checkout Integration:** Dedicated self-scanning kiosk APIs.

3.4 Won't Have: Out-of-Scope (Delimitations)

- Physical automated robotic cranes, Automated Guided Vehicles (AGVs), or mechanized sortation conveyor belts.
- Full double-entry corporate accounting / financial general ledger (system integrates via clean REST APIs to external ERPs).
- Custom silicon / ASIC hardware fabrication.
- International cross-border customs bond clearing workflows.

4. Detailed Feature Specifications & User Stories

4.1 Module M-01: Authentication, Access Control & Security

- **FR-01 (JWT Authentication):** The system shall authenticate users via email/username and password, issuing cryptographically signed JSON Web Tokens (access token valid for 15 minutes, refresh token valid for 8 hours).
- **FR-02 (RBAC Enforcement):** The system shall enforce role-based access control isolating Cashier, Floor Operator, Receiving Clerk, Supervisor, Procurement Officer, Store Manager, and Administrator.
- **FR-04 (Terminal Session Timeout):** Mobile scanning terminals shall automatically lock after 15 minutes of inactivity to prevent unauthorized floor access.
- **User Story US-01:**

As a Warehouse Floor Operator,

I want to log in using my 4-digit PIN / credential on a shared mobile scanner,

So that I can immediately access my assigned putaway tasks without seeing unauthorized managerial or pricing data.

Acceptance Criteria (Gherkin):

- **Given** an operator with role **OPERATOR**,
- **When** the operator enters valid credentials on the mobile web portal,
- **Then** the system returns an access token restricted to **/api/v1/putaway** and **/api/v1/picking**,
- **And** any attempt to access **/api/v1/suppliers** or **/api/v1/finance** returns **HTTP 403 Forbidden**.

4.2 Module M-02 & M-03: Product Catalog & Supplier PO Lifecycle

- **FR-07 (Master SKU Management):** The system shall maintain unique SKU codes, EAN-13/GS1-128 barcodes, packaging dimensions, category hierarchy, and temperature classification.
- **FR-10 (Shelf-Life Rules):** The system shall enforce minimum receiving shelf-life rules (e.g., minimum 75% remaining shelf life required at dock).
- **FR-15 (Digital PO Creation):** Procurement officers shall create digital Purchase Orders linked to approved suppliers with delivery deadlines and unit purchase prices.
- **User Story US-02:**

As a Procurement Officer,

I want the system to generate a draft PO populated with the mathematically optimal EOQ when an item breaches its ROP,

So that I can review and approve supplier replenishments in under 30 seconds.

Acceptance Criteria (Gherkin):

- **Given** SKU **SKU-SOIL-1L** has an inventory position of 1,180 units and an ROP of 1,218 units,
- **When** the nightly replenishment cron or real-time sales trigger executes,
- **Then** a new PO in status **DRAFT** is created for supplier **Bashundhara Food & Beverage**,
- **And** the recommended order quantity is exactly set to the calculated EOQ (1,265 units).

4.3 Module M-04: Inbound Receiving & Digital GRN

- **FR-22 (Barcode Interrogation vs. PO):** The receiving clerk shall scan delivered carton/pallet barcodes to validate quantities and item identity against open PO line items.
- **FR-23 (Batch Lot & Expiry Capture):** The system shall require entering manufacturer batch numbers, production dates, and expiration dates for every perishable shipment.
- **FR-26 (Digital GRN Generation):** Upon receiving completion, the system shall generate a formal Goods Receipt Note (GRN) itemizing accepted units, short-shipped units, and damaged units.
- **User Story US-03:**

As an Inbound Receiving Clerk,

I want to record damaged cartons immediately during dock unloading,

So that damaged stock is quarantined and an automated credit note advisory is issued to the supplier.

Acceptance Criteria (Gherkin):

- **Given** PO [PO-2026-0891](#) specifies 100 cartons of milk,
- **When** the clerk enters 96 accepted cartons and 4 damaged/leaking cartons,
- **Then** GRN [GRN-2026-0412](#) is committed with 96 units marked [AVAILABLE](#) and 4 units marked [QUARANTINED](#),
- **And** the 4 quarantined units are excluded from sellable POS stock balances,
- **And** a credit advisory for 4 units is logged for supplier accounts reconciliation.

4.4 Module M-05 & M-06: Directed Putaway & Real-Time FEFO Inventory Ledger

- **FR-32 (Directed Spatial Putaway):** The system shall recommend the optimal bin destination based on SKU velocity (Class A near dispatch docks), temperature zones, and available bin cubic capacity.
- **FR-38 (Strict FEFO Expiry Queue):** The system shall strictly allocate outbound stock from the batch with the earliest valid expiration date.
- **FR-41 (Automated Quarantine Lock):** Any batch reaching its expiration date or manually flagged as damaged shall be mechanically locked from picking.
- **User Story US-04:**

As a Warehouse Picker,

I want my digital pick list to direct me strictly to the earliest expiring batch of yogurt,

So that we eliminate perishable waste and never dispatch expired stock to retail stores.

Acceptance Criteria (Gherkin):

- **Given** Bin B01 contains Batch [LOT-A](#) expiring in 6 days, and Bin B04 contains Batch [LOT-B](#) expiring in 25 days,
- **When** a store requisition requests 10 units of yogurt,
- **Then** the pick instruction directs the operator exclusively to Bin B01 for Batch [LOT-A](#),
- **And** scanning the barcode for Batch [LOT-B](#) triggers an audible error: "FEFO Violation: Pick Batch [LOT-A](#) First".

4.5 Module M-07 & POS: Sub-2.0s POS Concurrency Synchronization

- **FR-30 (Atomic Checkout Deduction):** When a retail cashier scans an item at checkout, the API shall deduct stock inside an atomic database transaction using row-level locking ([SELECT ... FOR UPDATE](#)).

- **FR-36 (Immutable Audit Ledger):** Every stock movement shall append a record to [inventory_transactions](#) with user ID, timestamp, transaction type, and reference number.
- **User Story US-05:**

As a Retail Cashier,

I want item barcode deductions to complete in under 2 seconds without database locking errors,

So that long customer checkout lines move quickly without delay.

Acceptance Criteria (Gherkin):

- **Given** 10 concurrent POS cash registers scanning items simultaneously across branches,
- **When** checkout API calls hit [/api/v1/pos/sync](#),
- **Then** 99% of requests complete with [HTTP 200 OK](#) in [< 1.2 seconds](#),
- **And** the inventory on-hand balance matches total initial stock minus total sold units exactly, with zero negative stock states.

5. Non-Functional Requirements (NFRs) & Performance SLOs

ID	Requirement Category	Specific Metric & Quantitative Target	Verification Methodology
NFR-01	Scan Latency	Barcode interrogation to visual UI confirmation ≤ 350 milliseconds.	Automated browser performance profiler with Bluetooth HID trigger.
NFR-02	POS API Latency	Checkout deduction API response time ≤ 2.0 seconds (p95 ≤ 800 ms, p99 ≤ 1.5 s).	Locust load test simulating 50 concurrent requests/sec.
NFR-03	Concurrency Isolation	Handle at least 50 concurrent scanners and 10 POS cash registers with zero deadlocks.	Stress test suite with parallel asynchronous worker threads.
NFR-04	Data Integrity	Complete ACID transaction compliance; zero instances of negative stock balances.	Multi-threaded race condition tests on last remaining stock unit.
NFR-05	System Availability	99.8% operational uptime during warehouse operational hours (6:00 AM to 11:00 PM BST).	Automated health-check monitoring via Prometheus / Uptime Kuma.
NFR-06	Disaster Recovery	Point-In-Time Recovery (PITR) with Recovery Point Objective (RPO) ≤ 1 min, RTO ≤ 15 min.	Simulated database crash and WAL recovery drills.
NFR-07	Security & Encryption	TLS 1.3 encryption in transit; AES-256 for credentials at rest; Argon2id password hashing.	SSL Labs SSL test (A+ grade) and OWASP ZAP penetration scan.

ID	Requirement Category	Specific Metric & Quantitative Target	Verification Methodology
NFR-08	Ergonomics & Display	High-contrast UI (WCAG 2.1 AA compliant); one-handed mobile navigation; audio/haptic beeps.	Usability field trial with warehouse operators under 150 lux light.
NFR-09	Scalability	Support up to 50,000 SKUs, 500,000 active batches, and 5,000,000 transaction records.	Database benchmark with 5M synthetic records using B-Tree indexes.
NFR-10	Regulatory Compliance	100% adherence to Bangladesh Food Safety Act 2013 and BSTI batch expiration standards.	Automated audit verifying zero expired units ever dispatched.

6. Edge Cases, Failure Modes & Resilience Protocols

6.1 Edge Case 1: The Zero-Stock Concurrency Collision

- **Scenario:** Two cashiers at different retail branches scan the very last available unit of full-cream milk at the exact same millisecond.
 - **System Action:**
- Both transactions request a row-level lock on the active inventory batch via `SELECT current_quantity FROM product_batches WHERE batch_id = :id FOR UPDATE`.
 - Transaction A acquires the lock first, decrements quantity from 1 to 0, commits, and releases the lock.
 - Transaction B acquires the lock, detects `current_quantity = 0`, immediately releases the lock, and returns `HTTP 409 Conflict: {"error": "OUT_OF_STOCK", "sku": "MILK-FC-1L"}`.
 - Cashier B's POS notifies the cashier gracefully; inventory never drops below zero.

6.2 Edge Case 2: Store Internet Drop (Network Outage)

- **Scenario:** The local ISP fiber connection at the Dhanmondi retail outlet is severed for 10 minutes during peak hours.
 - **System Action:**
- The POS PWA detects network failure and toggles to **Local Resilient Mode**.
 - Scanned sales are appended to an encrypted browser `IndexedDB` transaction buffer with client timestamps.
 - A visual amber badge alerts the cashier: `"Offline Mode: 14 Sales Buffered"`.
 - When connection is restored, the client executes an idempotent bulk sync (`POST /api/v1/pos/bulk-sync`) with `X-Idempotency-Key` headers. The central server validates and commits all 14 sales chronologically.

6.3 Edge Case 3: Damaged or Smudged Barcode Label

- **Scenario:** An incoming carton has its manufacturer barcode torn or smudged with oil.
 - **System Action:**
- Operator toggles mobile scanner to "Manual SKU / Batch Lookup".
 - Typing the first 3 letters of the product name returns autocomplete suggestions with product thumbnail images.
 - Clerk selects product, confirms batch expiry from legible text, and clicks "Print Replacement Barcode".

39. Connected dock label printer instantly outputs a fresh GS1-compliant internal barcode sticker.

7. Business KPIs & Operational Success Metrics

Strategic Metric	Baseline (Traditional Manual Super Shop)	RetailSync Target (Pilot Benchmark)	Business Impact & Financial Value
Perishable Spoilage Rate	15% to 22% of dairy/produce wasted annually.	< 6% (Over 65% waste reduction).	Saves millions of BDT in prevented stock write-offs.
Stockout Frequency (Class A SKUs)	7.5% to 11.2% lost sales from empty shelves.	< 1.5% (80% stockout reduction).	Immediate gross revenue expansion during peak shopping hours.
Inbound Unloading-to-GRN Cycle	45 to 75 minutes per delivery truck.	< 12 minutes (73% cycle time reduction).	Eliminates dock congestion and supplier waiting detention fees.
Bin Location Accuracy	78% to 84% (Frequent lost pallets and misplaced stock).	> 99.5% spatial location accuracy.	Eliminates picker search latency and delayed store deliveries.
Store Dispatch Accuracy	3.2% error rate (Store receives wrong quantities/items).	< 0.2% shipment discrepancy rate.	Eliminates store-warehouse disputes and inter-branch inventory leakage.
Inventory Turnover Ratio (ITR)	6.2 turns per year.	9.5+ turns per year.	Drastically lowers capital lockup in holding inventory.

MODULE 2: SOFTWARE ARCHITECTURE DOCUMENT (SAD)

Software Architecture Document (SAD)

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Technical Architecture, Component Decomposition, Concurrency Isolation, and Data Flow Design

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

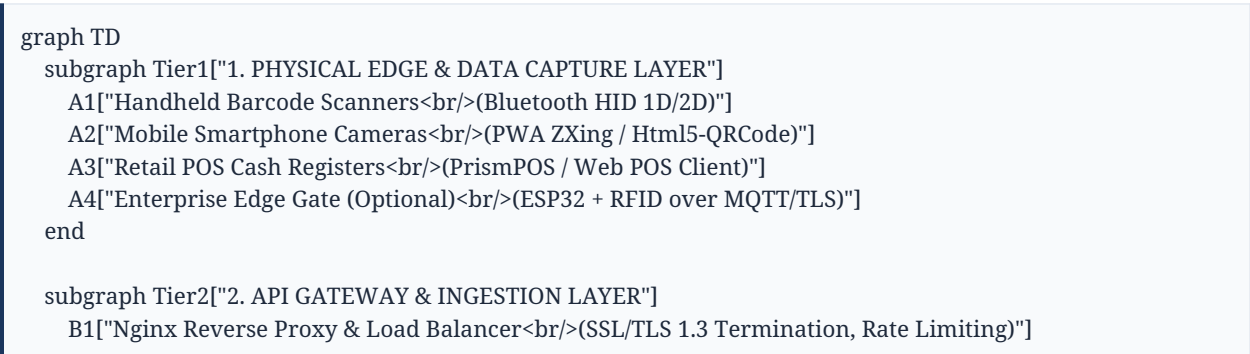
Date: September 2026 | Version: 1.0.0-RELEASE

1. Architectural Principles & System Goals

RetailSync is engineered as a **decoupled, multi-tier cyber-physical software platform** optimized for transactional reliability, sub-second latency, and rigorous data consistency:

- **ACID-First Relational Ledger:** Financial transactions and stock balance state transitions are non-negotiable. Every stock movement is logged in an append-only double-entry ledger within strict database transaction boundaries.
- **Sub-2.0s Concurrency Guarantee:** Retail POS checkout counters must never wait on warehouse database contention. High-throughput row-level locking (**SELECT ... FOR UPDATE**) guarantees zero deadlocks and zero phantom over-selling.
- **Decoupled Stateless Services:** Backend business domain services remain stateless, containerized via Docker, allowing horizontal scaling behind reverse proxies and connection poolers.
- **Offline-Tolerant Client Edges:** Network outages in steel-reinforced warehouse racking or retail branches are mitigated via client-side Service Workers and encrypted IndexedDB queues with idempotent sync replay.
- **Defense-in-Depth Security:** End-to-end TLS 1.3 encryption, Argon2id password hashing, JWT stateless access tokens with rotated refresh tokens, and immutable audit logs.
- **Hybrid Enterprise Edge Extensibility:** Native architectural adapters allow plugging in automated stationary dock gate scanners (ESP32 MCU + RFID / fixed industrial scanners) for bulk pallet intake.

2. High-Level 4-Tier System Architecture



```

    B2["Stateless API Gateway<br/>(JWT Auth, Request Validation, CORS)"]
    B3["MQTT Ingestion Broker<br/>(Mosquitto/EMQX with TLS & ACLs)"]
end

subgraph Tier3["3. BUSINESS APPLICATION & ENGINE LAYER"]
    C1["Inbound Receiving & GRN Service"]
    C2["Directed Spatial Putaway Engine"]
    C3["Real-Time FEFO/FIFO Ledger Service"]
    C4["Sub-2.0s POS Concurrency Controller"]
    C5["Replenishment DSS (EOQ / Greasley)"]
    C6["Outbound Wave & Pickpath Dispatch"]
    C7["Cycle Counting & Anomaly Detection"]
end

subgraph Tier4["4. PERSISTENCE & DATA STORAGE LAYER"]
    D1["PostgreSQL 16 Core Relational DB<br/>3NF Schema, ACID Ledger, B-Tree Indexes"]
    D2["Redis 7 In-Memory Cache<br/>Session Store, Distributed Locks, Rate Limits"]
    D3["Write-Ahead Logs (WAL) & PITR Storage<br/>Encrypted Cloud Object Storage / S3"]
end

A1 --> |Bluetooth / HTTPS| B1
A2 --> |HTTPS REST| B1
A3 --> |HTTPS POS Sync API| B1
A4 --> |MQTT Port 8883| B3

B1 --> B2
B3 --> C1
B2 --> C1
B2 --> C2
B2 --> C3
B2 --> C4
B2 --> C5
B2 --> C6
B2 --> C7

C1 & C2 & C3 & C4 & C5 & C6 & C7 --> D1
C4 & C2 --> D2
D1 -.-> |Continuous Archival| D3

```

3. Component & Service Decomposition

3.1 Service Catalog & Responsibilities

Service Identifier	Technology & Framework	Primary Architectural Responsibilities	Inter-Service Communication
Auth & Security Service	Node.js (TypeScript) / Python FastAPI	JWT token generation, refresh token rotation, Argon2id password hashing, RBAC permission evaluation.	Synchronous REST / Internal Middleware
Product Master Service	Python FastAPI / Pydantic v2	Master SKU catalog, EAN-13 barcode validation, ABC turnover classification, temperature requirements.	Synchronous REST, Redis Cache
Inbound & GRN Service	Python FastAPI / SQLAlchemy 2.0	Purchase Order matching, carton scan verification, damage quarantine segregation, digital GRN generation.	REST / PostgreSQL Transactions
Spatial Putaway Engine	Python FastAPI / NetworkX	Dynamic Zone-Aisle-Rack-Shelf-Bin mapping, volumetric/weight capacity evaluation, velocity-based bin recommendations.	REST / Redis Spatial Cache
FEFO Inventory Ledger	PostgreSQL 16 Stored Triggers / Python	Append-only transaction logging, batch expiry monitoring, FEFO priority queue sorting, automated quarantine locks.	ACID PostgreSQL Database Engine
POS Concurrency Controller	Python FastAPI / asyncpg	High-speed checkout deduction, row-level locking (SELECT ... FOR UPDATE), idempotency token validation.	Direct PostgreSQL connection pool
Replenishment DSS Engine	Python / NumPy / SciPy	Continuous calculation of EOQ, Greasley's Statistical Safety Stock, dynamic ROP alerting, automated draft PO creation.	Background Celery Worker / Cron
Outbound Wave Dispatch	Python FastAPI	Branch requisition consolidation, wave picking batching, shortest-path aisle routing (TSP heuristic), dispatch chalans.	REST / PostgreSQL Transactions

Service Identifier	Technology & Framework	Primary Architectural Responsibilities	Inter-Service Communication
Audit & Anomaly Service	Python / Scikit-learn	Immutable audit logging, cycle count blind entry reconciliation, Isolation Forest machine learning for shrinkage detection.	Asynchronous Celery Tasks

4. End-to-End Sequence Diagrams

4.1 Sequence 1: Inbound Receiving & Digital GRN Generation

```
sequenceDiagram
    autonumber
    actor Clerk as Receiving Clerk
    participant Scanner as Mobile PWA Scanner
    participant Gateway as API Gateway (Nginx)
    participant InboundSvc as Inbound & GRN Service
    participant DB as PostgreSQL Core DB

    Clerk->>Scanner: Selects PO #PO-2026-0891
    Scanner->>Gateway: GET /api/v1/inbound/po/PO-2026-0891
    Gateway->>InboundSvc: Query PO Line Items
    InboundSvc->>DB: SELECT * FROM purchase_orders WHERE po_number = ...
    DB-->>InboundSvc: Returns Expected Items (e.g. 100 Cartons Milk)
    InboundSvc-->>Scanner: Renders PO Checklist UI

    loop For Each Delivered Pallet/Carton
        Clerk->>Scanner: Scans Barcode (EAN-13) + Inputs Expiry Date
        Scanner->>InboundSvc: POST /api/v1/inbound/verify-scan {sku, batch, expiry, qty}
        InboundSvc->>InboundSvc: Validate Minimum Remaining Shelf-Life (>= 75%)
        InboundSvc-->>Scanner: Audio Beep (Green) + Accepted Count Incremented
    end

    opt Damage Detected
        Clerk->>Scanner: Flags 4 cartons as "Damaged" with photo
        Scanner->>InboundSvc: POST /api/v1/inbound/flag-damage {qty: 4, reason: "LEAKING"}
    end

    Clerk->>Scanner: Clicks "Complete Goods Receipt"
    Scanner->>InboundSvc: POST /api/v1/inbound/grn/complete
    InboundSvc->>DB: BEGIN TRANSACTION
    InboundSvc->>DB: INSERT INTO goods_receipt_notes (accepted=96, rejected=4)
    InboundSvc->>DB: INSERT INTO product_batches (status='AVAILABLE', qty=96)
    InboundSvc->>DB: INSERT INTO product_batches (status='QUARANTINED', qty=4)
    InboundSvc->>DB: INSERT INTO inventory_transactions (type='INBOUND_GRN', ...)
    InboundSvc->>DB: UPDATE purchase_orders SET status='COMPLETED'
```

```

InboundSvc-->>DB: COMMIT
InboundSvc-->>Scanner: Returns GRN #GRN-2026-0412 + Auto Credit Note
Scanner-->>Clerk: Displays GRN Summary & Triggers Pallet Label Print

```

4.2 Sequence 2: Sub-2.0s POS Checkout Atomic Stock Deduction

```

sequenceDiagram
    autonumber
    actor Cashier as Retail Store Cashier
    participant POS as Store POS Terminal
    participant Gateway as API Gateway
    participant POSController as POS Concurrency Controller
    participant DB as PostgreSQL Core DB

    Cashier->>POS: Scans Item Barcode (e.g., 8941100234123)
    POS->>Gateway: POST /api/v1/pos/sync<br/>Headers: X-Idempotency-Key: uuid-98234<br/>Payload:
    {register_code: "REG-01", barcode: "8941100234123", qty: 2}
    Gateway->>POSController: Process Atomic Deduction
    POSController->>DB: BEGIN TRANSACTION (Isolation: READ COMMITTED)

    critical Row-Level Locking on Earliest FEFO Batch
        POSController->>DB: SELECT batch_id, current_quantity, expiry_date<br/>FROM
        product_batches<br/>WHERE product_id = :p_id AND quarantine_status = 'AVAILABLE' AND current_quantity
        >= :qty<br/>ORDER BY expiry_date ASC LIMIT 1<br/>FOR UPDATE;
        DB-->>POSController: Returns Batch #B-4091 (Current Qty: 45, Expiry: 2026-10-04)
        POSController->>DB: UPDATE product_batches<br/>SET current_quantity = current_quantity -
        2<br/>WHERE batch_id = 'B-4091';
        POSController->>DB: INSERT INTO inventory_transactions<br/>(product_id, batch_id,
        type='POS_SALE_FEFO', qty=-2, register_id=...);
    end

    POSController->>DB: COMMIT;
    DB-->>POSController: Transaction Committed in 18ms
    POSController-->>POS: HTTP 200 OK {status: "CONFIRMED", remaining_stock: 43} [Elapsed: 140ms]
    POS-->>Cashier: Item Added to Customer Bill Instantly

```

5. Concurrency Control & Transaction Isolation Architecture

5.1 The Concurrency Challenge in Super Shop Retail

During festival rush periods (such as Ramadan afternoon grocery shopping), multiple cash registers across branches and warehouse pickers simultaneously query and decrement stock balances. Naive optimistic locking (`UPDATE ... WHERE version = x`) results in widespread transaction rollback errors on high-velocity items, frustrating cashiers and stalling checkout queues.

5.2 Pessimistic Row-Level Locking Architecture (`SELECT ... FOR UPDATE`)

RetailSync enforces **Pessimistic Row-Level Locking** specifically at the batch row level inside an atomic transaction:

```
-- RetailSync Atomic Deduction Stored Procedure / Query Pattern
BEGIN;

-- 1. Identify and lock exclusively the earliest active FEFO batch with adequate stock
SELECT batch_id, current_quantity
FROM product_batches
WHERE product_id = $1
  AND quarantine_status = 'AVAILABLE'
  AND current_quantity >= $2
ORDER BY expiry_date ASC
LIMIT 1
FOR UPDATE;

-- 2. Decrement physical stock
UPDATE product_batches
SET current_quantity = current_quantity - $2
WHERE batch_id = $selected_batch_id;

-- 3. Append immutable transaction audit record
INSERT INTO inventory_transactions (
  product_id, batch_id, location_id, transaction_type, quantity, reference_document_no, user_id
) VALUES (
  $1, $selected_batch_id, $location_id, 'POS_SALE_FEFO', -$2, $receipt_no, $user_id
);

COMMIT;
```

5.3 Deadlock Elimination Strategy

40. **Deterministic Lock Ordering:** If a composite transaction deducts multiple SKUs (e.g., shopping cart checkout), the application layer strictly sorts all requested SKU IDs in ascending numerical order before acquiring database locks (`ORDER BY product_id ASC`).
41. **Short Transaction Life:** Business logic validation (price calculation, customer loyalty) executes *prior* to opening the database transaction. Database transactions remain open for less than **25 milliseconds**.
42. **Connection Pooling Tuning:** PgBouncer maintains dedicated transaction-mode pools ensuring high throughput without exhausting database worker threads.

6. Offline-First Resilience & Network Outage Architecture

```
graph TD
  subgraph ClientEdge["Retail Branch POS / Mobile Scanner Edge"]
    UI["React / PWA UI Layer"]
    SW["Service Worker (Cache Storage)"]
    IDB["Encrypted IndexedDB<br/>Offline Transactions Queue"]
    SyncManager["Background Sync Manager"]
  end
```

```

end

subgraph Network["Connectivity Layer"]
  ISP["Internet Link (Fiber / 4G)"]
end

subgraph CentralServer["RetailSync Central Cloud Server"]
  API["Idempotent POS Ingestion API"]
  IdemCache["Redis Idempotency Cache<br/>(Key: X-Idempotency-Key)"]
  CoreDB["PostgreSQL ACID Core"]
end

UI --> | 1. Scan Item | SW
SW --> | 2. Check Connectivity | ISP
ISP -. -> | 3a. Connection Failed / Drop | IDB
SW --> | 3b. Buffer Transaction | IDB
UI --> | Visual Amber Badge | UI

SyncManager --> | 4. Detects Link Recovery | IDB
SyncManager --> | 5. Bulk Replay with Idempotency Tokens | ISP
ISP --> | POST /api/v1/pos/bulk-sync | API
API --> | 6. Check Key Exists? | IdemCache
IdemCache --> | New Key | CoreDB
CoreDB --> | 7. Commit Batch | API
API --> | HTTP 200 Success | SyncManager
SyncManager --> | 8. Clear Buffered Queue | IDB

```

6.1 Idempotency Guarantee

Every offline transaction generates a unique UUIDv4 token passed via the HTTP header:

X-Idempotency-Key: e4b2d184-7832-4e92-9844-329810a911ab

When bulk-sync replays, Redis verifies whether the idempotency key was previously processed within a 48-hour TTL, completely eliminating duplicate stock deductions even if network retries occur multiple times.

7. Security, RBAC & Audit Architecture

```

graph LR
  subgraph Client["Client Tier"]
    ClientApp["Browser / PWA / Scanner"]
  end

  subgraph SecurityGate["Security & Auth Middleware"]
    TLS["TLS 1.3 Encryption<br/>(Port 443)"]
    JWTVerify["JWT Signature Verification<br/>(HMAC-SHA256 / Ed25519)"]
    RBAC["Role-Based Access Guard<br/>(Roles: Cashier, Clerk, Operator, Sup, Proc, Admin)"]
  end

```

```

subgraph Resources["Protected Domain Endpoints"]
  PublicEnd["Public: /api/v1/auth/login"]
  OperatorEnd["Floor: /api/v1/putaway, /picking"]
  ManagerEnd["Management: /api/v1/po, /adjustments"]
  AdminEnd["Admin: /api/v1/admin, /system"]
end

subgraph Audit["Audit Layer"]
  AuditLogger["Append-Only Security & Audit Logger"]
  DBAudit["PostgreSQL<br/>inventory_transactions & security_logs"]
end

ClientApp --> TLS
TLS --> JWTVerify
JWTVerify --> RBAC
RBAC --> OperatorEnd
RBAC --> ManagerEnd
RBAC --> AdminEnd
JWTVerify --> PublicEnd

OperatorEnd & ManagerEnd & AdminEnd --> AuditLogger
AuditLogger --> DBAudit

```

7.1 Database Permission Hardening

At the database engine level, the application database user role (`retailsync_app`) is granted `INSERT` and `SELECT` privileges on `inventory_transactions`, but is strictly revoked of `UPDATE` and `DELETE` permissions:

```

REVOKE UPDATE, DELETE ON inventory_transactions FROM retailsync_app;
-- Guaranteed immutable audit ledger by engine constraints

```

8. Hybrid Enterprise Edge Gateway (WarePulse IoT Integration)

For high-throughput central distribution warehouses receiving 50+ pallet deliveries daily, RetailSync integrates the **WarePulse IoT Edge Gateway** as an optional high-speed automated dock intake portal:

```

graph TD
  subgraph DockPortal["Central Distribution Dock Gate Array"]
    Gate["Overhead Fixed Dock Portal"]
    MCU["ESP32 Dual-Core 240MHz Microcontroller"]
    RFID["RC522 High-Frequency RFID Reader / Fixed GS1 Barcode Gun"]
    Flash["LittleFS Non-Volatile Flash Cache (Up to 10,000 Scans)"]
    Status["Audio-Visual Status Beeper / LED"]
  end

  subgraph Telemetry["Encrypted Telemetry Pipeline"]
    MQTT["MQTT Broker (Mosquitto/EMQX)<br/>TLS 1.3, Port 8883, X.509 Client Certs"]
  end

```

```

    Topic["Topic: warehouse/dock-01/inbound (QoS 1)"]
end

subgraph RetailSyncCore["RetailSync Core Platform"]
    IngestionWorker["Dock Ingestion Worker Service"]
    POEngine["Purchase Order Auto-Reconciler"]
    CoreDB["PostgreSQL 16 DB"]
end

Gate --> |Pallet Passes Under Dock| RFID
RFID --> |SPI Bus| MCU
MCU --> Status
MCU --> |Network Brownout| Flash
Flash --> |On Link Reconnect| MCU
MCU --> |JSON Payload over MQTTs| MQTT
MQTT --> Topic
Topic --> IngestionWorker
IngestionWorker --> POEngine
POEngine --> CoreDB

```

- **Zero-Touch Inbound Registration:** Pallets pass through loading dock portals; tags are read hands-free within 150 ms; draft GRNs are created automatically in RetailSync with zero manual data entry.

MODULE 3: TECHNOLOGY STACK & ARCHITECTURAL DECISION RECORDS

Technology Stack & Architecture Decision Records (ADR)

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Engineering Stack Selection, Component Standards, and Architectural Decision Records

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Curated Technology Stack Matrix

RetailSync is built using modern, open-source, battle-tested software engineering technologies carefully curated for high throughput, sub-second latency, developer productivity, and near-zero software licensing costs:

RETAILSYNC STACK	
FRONTEND PWA	Next.js 14 / React 18, Tailwind CSS, TanStack Query, Zustand
BARCODE ENGINES	ZXing / Html5-QRCode + Bluetooth HID Hardware Scanner Profiles
BACKEND API	Python 3.11+ / FastAPI (Asynchronous ASGI), Pydantic v2
DATABASE CORE	PostgreSQL 16 (ACID 3NF Normalized, Composite B-Tree Indexes)
IN-MEMORY CACHE	Redis 7 (Idempotency Keys, Session Store, Distributed Rate Locks)
DECISION ENGINE	NumPy, SciPy (Greasley Variance, EOQ), Scikit-learn (Isolation)
INFRA & DEVOPS	Docker, Docker Compose, Nginx Reverse Proxy, GitHub Actions CI/CD
TESTING FRAMEWORKS	Pytest, HTTPX AsyncClient, Locust (Concurrency Stress Testing)

1.1 Detailed Technology Specifications

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
Frontend Framework	Next.js / React	14.2+ (App Router)	Single Page Application (SPA) speed with server-side rendering (SSR) for dashboards; eliminates page-reload latency on mobile scanning terminals.

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
Styling & Design System	Tailwind CSS	3.4+	Rapid component styling; ultra-clean modern aesthetic; utility-first layout supporting high-contrast floor themes.
Client State Management	Zustand	4.5+	Extremely lightweight (1.1 kB), minimalist state store for managing active picker sessions, scan queues, and cart states without Redux boilerplate.
Data Fetching & Cache	TanStack Query	v5.0+	Automatic background refetching, query caching, and optimistic UI updates for real-time warehouse bin occupancy rendering.
Barcode Scanner Integration	Html5-QRCode / ZXing	Latest Stable	Pure JavaScript barcode engine reading 1D (EAN-13, Code 128, GS1) and 2D (QR) barcodes directly from smartphone cameras and Bluetooth trigger guns.
Backend API Framework	Python FastAPI	0.111+ (Python 3.11)	High-performance asynchronous execution on Starlette/uvloop; automatic OpenAPI 3.1 / Swagger documentation; deep integration with scientific math libraries.
Data Validation & Typing	Pydantic	v2.7+ (Rust core)	Blazing-fast request payload parsing and validation with strict type enforcement for financial quantities and barcode payloads.
Database ORM & Driver	SQLAlchemy + asyncpg	2.0+ / 0.29+	Asynchronous database access with connection pooling; raw SQL query performance for complex row-locking operations.
Core Relational Database	PostgreSQL	16.3	World-class ACID transaction guarantees, robust row-level locking (SELECT ... FOR UPDATE), JSONB column support, and Point-In-Time Recovery (PITR).

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
In-Memory Cache & Key-Store	Redis	7.2	Ultra-fast in-memory key-value store for API rate-limiting, idempotency tokens (X-Idempotency-Key), and ephemeral session storage.
Replenishment Algorithms	NumPy / SciPy	1.26+ / 1.13+	High-precision vector math for calculating Greasley's dual-variance safety stock and statistical Z-score quantiles.
Shrinkage Anomaly Detection	Scikit-learn	1.4+	Unsupervised IsolationForest model to detect anomalous stock discrepancy clusters across shifts and zones.
Reverse Proxy & Gateway	Nginx	1.26 (Alpine)	High-performance reverse proxy handling TLS 1.3 termination, gzip compression, static asset serving, and request rate-limiting.
Containerization	Docker & Compose	26.0+ / v2.27+	Guaranteed development-to-production parity; single-command local multi-container spin-up (docker compose up -d).
Concurrency Load Testing	Locust	2.28+	Distributed Python-based load testing simulating 50+ concurrent cashier POS deductions to prove sub-2.0s SLA compliance.

2. Architecture Decision Records (ADRs)

ADR-01: Relational PostgreSQL vs. NoSQL MongoDB

- **Status:** ACCEPTED
 - **Context:** The system manages physical inventory, financial purchase orders, supplier credit notes, and store billing transactions.
 - **Decision:** We select **PostgreSQL 16** as the core database engine instead of a NoSQL document database (like MongoDB).
 - **Rationale:**
43. *ACID Compliance & Ledger Integrity:* A warehouse and POS system requires strict ACID properties. Stock deduction and financial ledger updates cannot afford eventual consistency anomalies where units "disappear" or negative stock is accepted.

44. *Relational Constraints*: Enforcing referential integrity (e.g., preventing deletion of an active SKU when inventory batches exist via **ON DELETE RESTRICT**) is handled natively by the database engine, preventing orphaned records.
45. *Row-Level Locking*: PostgreSQL offers world-class row-level locking (**SELECT ... FOR UPDATE**) essential for resolving concurrent cashier checkouts.
- **Consequences**: Schema changes require structured migrations (via Alembic). Horizontal sharding is more complex than NoSQL, but acceptable given that a single normalized PostgreSQL instance with B-Tree indexes easily supports tens of millions of records.

ADR-02: Pessimistic Row-Level Locking vs. Optimistic Concurrency Control

- **Status**: ACCEPTED
 - **Context**: High-velocity FMCG items (e.g., 1-Litre Soyabean Oil, Milk Vita cartons) experience multiple simultaneous checkout deductions across different retail registers.
 - **Decision**: We implement **Pessimistic Row-Level Locking** (**SELECT ... FOR UPDATE**) for stock deductions, rather than Optimistic Concurrency Control (OCC with version column).
 - **Rationale**:
46. *Retry Storms in OCC*: Under high contention (such as a festival rush with 5 cashiers selling the same fast-moving item simultaneously), OCC causes 4 out of 5 transactions to fail and trigger application-layer retries. This creates compounding latency spikes and risks user frustration.
 47. *Deterministic Queueing*: Pessimistic row locking briefly serializes only the specific batch row for < 25 ms. Each cashier transaction waits a few milliseconds, decrements stock, and completes cleanly without rollback exceptions.
 - **Consequences**: Requires disciplined lock ordering (sorting requested SKUs in ascending order) to completely prevent deadlock scenarios.

ADR-03: Progressive Web Application (PWA) vs. Native Android/Kotlin APK

- **Status**: ACCEPTED
 - **Context**: Warehouse floor operators and receiving clerks need mobile barcode scanning interfaces on handheld terminals.
 - **Decision**: We build the mobile operator interface as a **Progressive Web Application (PWA)** in Next.js/React, rather than a native Android Kotlin application.
 - **Rationale**:
48. *Frugal Hardware Agility*: Industrial Android terminals (Zebra, Honeywell) cost upwards of 60,000 BDT. A responsive PWA runs smoothly on standard consumer smartphones (Samsung, Xiaomi, Symphony) paired with 3,500 BDT Bluetooth barcode trigger handles.
 49. *Zero App-Store Friction & Instant Updates*: Warehouse floor workers do not need to install APKs or configure MDM systems. Any bug fix or algorithm update deployed to the web server is instantly available to all floor devices upon refresh.
 50. *Offline Resilience via Service Workers*: Modern Web APIs (Cache Storage API and IndexedDB) provide robust offline capabilities comparable to native SQLite storage.
 - **Consequences**: Camera barcode scanning in low-light environments requires high-quality camera autofocus, which is mitigated by supporting physical Bluetooth HID scanner grips.

ADR-04: Python FastAPI vs. Node.js Express for Core Backend

- **Status:** ACCEPTED
- **Context:** The backend must handle high-concurrency I/O (barcode scans and POS sync) while simultaneously executing complex mathematical operations (Greasley's Safety Stock, EOQ, and Machine Learning anomaly detection).
- **Decision:** We select **Python 3.11+ with FastAPI** as the primary backend framework.
- **Rationale:**
 51. *Scientific & Algorithmic Ecosystem:* Implementing Greasley's dual-variance formula and Isolation Forest anomaly models in Node.js requires awkward third-party wrappers or spawning Python subprocesses. FastAPI natively executes NumPy, SciPy, and Scikit-learn within the same runtime.
 52. *Asynchronous High-Throughput I/O:* FastAPI running on **uvloop** matches or exceeds Node.js Express throughput for I/O-bound database queries.
 53. *Pydantic v2 Type Safety:* Automatic request validation, serialization, and interactive Swagger UI documentation out of the box.
- **Consequences:** Developers must write asynchronous code (**async/await**) properly to avoid blocking the event loop during heavy CPU calculations (heavy math is offloaded to Celery background workers).

ADR-05: Stateless JWT Tokens with Rotated Refresh Cookies

- **Status:** ACCEPTED
- **Context:** The system must securely authenticate hundreds of mobile scanner sessions, desktop management consoles, and retail POS cash registers.
- **Decision:** We implement a **Stateless Dual-Token Architecture**: Short-lived JWT Access Tokens (15 minutes) combined with HttpOnly, Secure, SameSite Refresh Tokens (8 hours) with automated refresh token rotation.
- **Rationale:**
 54. *Stateless API Gateway:* The API Gateway verifies token signatures locally without querying the database for every single barcode scan, guaranteeing sub-second latency.
 55. *Mitigation of XSS and CSRF:* Sensitive refresh tokens are stored in HttpOnly cookies unreachable by malicious client-side JavaScript, while access tokens reside in memory.
 56. *Immediate Revocation on Theft:* When a refresh token is used, it is rotated. If a compromised refresh token is reused, all tokens in that session family are immediately invalidated in Redis.
- **Consequences:** Clock synchronization (NTP) across servers is required to ensure consistent JWT expiry validation.

3. Infrastructure & Deployment Environment Specifications

3.1 Local Development Environment

- **OS:** Linux (Ubuntu 22.04+ / Debian 12) or macOS / Windows WSL2.
- **Runtimes:** Python **3.11.9+**, Node.js **18.20+** (LTS), Docker **26.0+**.
- **Database Tools:** Beekeeper Studio, pgAdmin 4, or DBeaver.
- **Hardware Requirements:** Minimum 8 GB RAM (16 GB recommended), 4-Core CPU, 20 GB free disk space.

3.2 Docker Compose Service Ports & Network Topology

```
version: '3.8'
```

```

services:
  retailsync-db:
    image: postgres:16-alpine
    container_name: retailsync-db
    environment:
      POSTGRES_DB: retailsync_db
      POSTGRES_USER: retailsync_user
      POSTGRES_PASSWORD: retailsync_secure_password_2026
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./docs/DATABASE_SCHEMA.sql:/docker-entrypoint-initdb.d/init.sql

  retailsync-redis:
    image: redis:7.2-alpine
    container_name: retailsync-redis
    ports:
      - "6379:6379"
    command: ["redis-server", "--appendonly", "yes"]

  retailsync-backend:
    build: ./backend
    container_name: retailsync-backend
    environment:
      DATABASE_URL: postgresql+asyncpg://retailsync_user:retailsync_secure_password_2026@retailsync-
db:5432/retailsync_db
      REDIS_URL: redis://retailsync-redis:6379/0
      JWT_SECRET: super_secure_dev_jwt_secret_key_diu_2026
    ports:
      - "8000:8000"
    depends_on:
      - retailsync-db
      - retailsync-redis

  retailsync-frontend:
    build: ./frontend
    container_name: retailsync-frontend
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000/api/v1
    ports:
      - "3000:3000"
    depends_on:
      - retailsync-backend

volumes:
  pgdata:

```

MODULE 4: RELATIONAL DATABASE SCHEMA & DATA DICTIONARY

Relational Database Schema & Data Dictionary Specification

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Normalized 3NF Schema Architecture, Data Dictionary, Constraints, Triggers, and DDL Specifications

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Schema Architecture & Design Philosophy

The RetailSync database is architected on **PostgreSQL 16+** with strict **Third Normal Form (3NF)** relational normalization. The data model is designed to support high-throughput, low-latency concurrent operations, sub-2.0s Point of Sale (POS) inventory deductions, and mathematical inventory replenishment models.

1.1 Key Architectural Principles

- **Atomic Double-Entry Inventory Ledger:** Inventory balances are backed by an immutable ledger of transactions ([inventory_transactions](#)), ensuring complete traceability of all stock movements.
- **Granular Spatial Hierarchy:** Warehouses, zones, aisles, racks, shelves, and bins are modeled hierarchically in the [locations](#) table to facilitate algorithmic putaway and pick path optimization.
- **Perishable FEFO Tracking:** The [product_batches](#) table models batch numbers, manufacturing dates, and expiration dates as first-class attributes, enabling automated First-Expired, First-Out picking logic.
- **Concurrency Isolation:** Row-level locks (`SELECT ... FOR UPDATE`) are applied on [product_batches](#) during POS deduction transactions to eliminate race conditions and prevent negative stock.
- **Referential Integrity & Cascading Policies:** Foreign key relationships enforce referential integrity with strict `RESTRICT` rules on critical entities to prevent accidental data loss.

2. Entity-Relationship (ER) Architecture

The entity relationships are structured into five core functional clusters:

RETAILSYNC RELATIONAL CLUSTERS		
1. Master Data & RBAC	users, suppliers, products	
2. Spatial Warehouse	locations (Aisles, Racks, Shelves, Bins)	
3. Batch & Ledger Core	product_batches, inventory_transactions	
4. Inbound & Procurement	purchase_orders, po_line_items, goods_receipt_notes	
5. Outbound & Retail POS	branch_stores, pos_registers, store_requisitions, requisition_items	
6. Quality & Audit	cycle_counts, cycle_count_items	

+-----+

```
erDiagram
    users ||--o{ purchase_orders : creates
    users ||--o{ inventory_transactions : performs
    suppliers ||--o{ products : supplies
    suppliers ||--o{ purchase_orders : receives
    products ||--o{ product_batches : contains
    locations ||--o{ product_batches : stores
    purchase_orders ||--o{ po_line_items : contains
    purchase_orders ||--o{ goods_receipt_notes : fulfills
    products ||--o{ po_line_items : references
    product_batches ||--o{ inventory_transactions : logs
    branch_stores ||--o{ pos_registers : contains
    branch_stores ||--o{ store_requisitions : requests
    store_requisitions ||--o{ requisition_line_items : specifies
    cycle_counts ||--o{ cycle_count_items : audits
    product_batches ||--o{ cycle_count_items : verifies
```

3. Data Dictionary & Table Definitions

3.1 Table: `users`

Stores user identities, hashed credentials, and role-based access control (RBAC) assignments.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier for the user account
username	VARCHAR(50)	UNIQUE, NOT NULL	Alphanumeric login username
email	VARCHAR(100)	UNIQUE, NOT NULL	User email address
password_hash	VARCHAR(255)	NOT NULL	Argon2id cryptographic password hash
full_name	VARCHAR(100)	NOT NULL	Employee full legal name
role	user_role_enum	NOT NULL, DEFAULT 'OPERATOR'	System role (OPERATOR, CLERK, SUPERVISOR, etc.)
phone_number	VARCHAR(20)	NULLABLE	Contact telephone number
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active/disabled status flag
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Column Name	Data Type	Constraints	Description
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Last profile update timestamp

3.2 Table: `suppliers`

Maintains verified vendor profiles, payment terms, and vendor reliability metrics.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique supplier identifier
code	VARCHAR(20)	UNIQUE, NOT NULL	Business vendor code (e.g., SUP-PRAN-001)
name	VARCHAR(150)	NOT NULL	Corporate legal trading name
contact_person	VARCHAR(100)	NULLABLE	Primary representative name
phone	VARCHAR(20)	NOT NULL	Primary contact phone number
email	VARCHAR(100)	NULLABLE	Official correspondence email
address	TEXT	NULLABLE	Physical office / warehouse address
lead_time_days	INTEGER	NOT NULL, DEFAULT 3	Historical average delivery lead time
rating	NUMERIC(3,2)	CHECK (rating BETWEEN 1 AND 5)	Vendor delivery performance score
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Operational active status

3.3 Table: `products`

The master catalog of SKUs sold across all retail supermarket branches.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Master product SKU identifier
sku	VARCHAR(50)	UNIQUE, NOT NULL	Internal Stock Keeping Unit code
barcode	VARCHAR(50)	UNIQUE, NOT NULL	EAN-13, UPC, or Code-128 retail barcode
name	VARCHAR(200)	NOT NULL	Full product commercial name

Column Name	Data Type	Constraints	Description
category	product_category_enum	NOT NULL	FMCG_DRY, PERISHABLE_PRODUCE, DAIRY, etc.
storage_temp	storage_temp_enum	NOT NULL, DEFAULT 'AMBIENT'	AMBIENT, CHILLED_COLD, DEEP_FROZEN
uom	VARCHAR(20)	NOT NULL, DEFAULT 'PIECE'	Unit of Measure (PIECE, KG, LITER, PACK)
unit_cost	NUMERIC(12,2)	NOT NULL, CHECK (unit_cost >= 0)	Procurement cost per unit in BDT
selling_price	NUMERIC(12,2)	NOT NULL, CHECK (selling_price >= unit_cost)	Retail checkout sales price in BDT
reorder_point	INTEGER	NOT NULL, DEFAULT 10	Algorithmic reorder threshold
safety_stock	INTEGER	NOT NULL, DEFAULT 5	Greasley statistical safety buffer
eoq_quantity	INTEGER	NOT NULL, DEFAULT 50	Economic Order Quantity batch target
shelf_life_days	INTEGER	NULLABLE	Product durability span in days

3.4 Table: `locations`

Models the physical warehouse spatial hierarchy down to discrete coordinate bins.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique storage location identifier
code	VARCHAR(50)	UNIQUE, NOT NULL	Barcode-scannable coordinate code (e.g., A01-R02-S03-B04)
zone	VARCHAR(20)	NOT NULL	Functional warehouse zone (DRY_STORAGE, CHILLER, RETAIL_FLOOR)
aisle	VARCHAR(10)	NOT NULL	Physical aisle designation
rack	VARCHAR(10)	NOT NULL	Vertical rack structure number
shelf	VARCHAR(10)	NOT NULL	Shelf tier level
bin	VARCHAR(10)	NOT NULL	Specific slot / bin coordinate
max_weight_kg	NUMERIC(10,2)	NOT NULL, DEFAULT 500.00	Maximum permissible load capacity
is_occupied	BOOLEAN	NOT NULL, DEFAULT FALSE	Dynamic occupancy state flag

3.5 Table: `product\_batches`

The foundational table enabling strict FEFO tracking, batch expiry isolation, and row locking.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique batch record identifier
batch_number	VARCHAR(50)	NOT NULL	Supplier / manufacturing batch code
product_id	UUID	NOT NULL, FK -> products(id)	Associated master product
location_id	UUID	NOT NULL, FK -> locations(id)	Storage location where batch is docked
manufacture_date	DATE	NOT NULL	Date of industrial production
expiry_date	DATE	NOT NULL	Date of product expiration
received_qty	INTEGER	NOT NULL, CHECK (received_qty > 0)	Original intake quantity received
current_qty	INTEGER	NOT NULL, CHECK (current_qty >= 0)	Available unreserved stock balance
status	batch_status_enum	NOT NULL, DEFAULT 'AVAILABLE'	AVAILABLE, QUARANTINED, EXPIRED, DEPLETED

4. Key Database Triggers & Automated Rules

4.1 Trigger 1: Automatic Location Occupancy Update

Whenever an intake or pick operation modifies `current_qty` in `product_batches`, an asynchronous database trigger computes the remaining aggregate stock in that bin and toggles `locations.is_occupied` automatically.

```
CREATE OR REPLACE FUNCTION fn_update_location_occupancy()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT COALESCE(SUM(current_qty), 0) FROM product_batches WHERE location_id = NEW.location_id) >
    0 THEN
        UPDATE locations SET is_occupied = TRUE WHERE id = NEW.location_id;
    ELSE
        UPDATE locations SET is_occupied = FALSE WHERE id = NEW.location_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_batch_location_occupancy
AFTER INSERT OR UPDATE OF current_qty ON product_batches
FOR EACH ROW EXECUTE FUNCTION fn_update_location_occupancy();
```

4.2 Trigger 2: Automated Perishable Expiry Quarantine

A scheduled PostgreSQL function executes daily to transition batches whose `expiry_date < CURRENT_DATE` to `EXPIRED`, and batches within 3 days of expiration to `QUARANTINED`, emitting an audit alert to supervisory dashboards.

```
CREATE OR REPLACE FUNCTION fn_auto_quarantine_expiring_batches()
RETURNS void AS $$
BEGIN
    UPDATE product_batches
    SET status = 'EXPIRED'
    WHERE expiry_date <= CURRENT_DATE AND status != 'EXPIRED';

    UPDATE product_batches
    SET status = 'QUARANTINED'
    WHERE expiry_date <= (CURRENT_DATE + INTERVAL '3 days')
    AND status = 'AVAILABLE';
END;
$$ LANGUAGE plpgsql;
```

5. Performance Indexing Strategy

To guarantee the **< 2.0s POS transaction completion SLA**, composite B-Tree indexes are deployed across critical search paths:

```
-- 1. Accelerates POS barcode lookups to sub-millisecond execution
CREATE INDEX idx_products_barcode ON products(barcode);

-- 2. Powers high-velocity FEFO sorting by product and nearest expiration
CREATE INDEX idx_batches_fefo ON product_batches(product_id, expiry_date ASC, current_qty)
WHERE status = 'AVAILABLE';

-- 3. Optimizes warehouse spatial scanning during barcode putaway
CREATE INDEX idx_locations_code ON locations(code);

-- 4. Speeds up real-time audit ledger reporting
CREATE INDEX idx_inventory_tx_created ON inventory_transactions(product_id, created_at DESC);
```

6. Seed Data & Benchmark Validation

The schema includes realistic Bangladeshi supermarket data covering FMCG staples (Radhuni Turmeric, Teer Soyabean Oil, Pran Frooto, Aarong Butter, etc.), 5 supplier profiles, 20 discrete warehouse bins across 3 climate zones, and 12 inventory batches verifying FEFO sorting order and quarantine state logic.

(Refer to `docs/DATABASE\_SCHEMA.sql` for the complete 600+ line production SQL script).

MODULE 5: REST API SPECIFICATION & ENDPOINT CONTRACTS

REST API Specification & Contract

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: RESTful OpenAPI 3.1 Aligned Endpoint Contracts, Data Schemas, and Response Envelopes

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Global API Conventions & Protocol Standards

1.1 Base URLs

- **Local Development:** <http://localhost:8000/api/v1>
- **Staging / Demonstration:** <https://staging-api.retailsync.com/api/v1>
- **Production Cluster:** <https://api.retailsync.com/api/v1>

1.2 Common HTTP Request Headers

```
Authorization: Bearer <JWT_ACCESS_TOKEN>
Content-Type: application/json
Accept: application/json
X-Client-Version: 1.0.0
X-Idempotency-Key: e4b2d184-7832-4e92-9844-329810a911ab (Mandatory for write/deduction endpoints)
```

1.3 Standard JSON Response Envelope

Every API response adheres to a predictable, strongly-typed envelope:

Success Response Envelope (HTTP 200 / 201)

```
{
  "success": true,
  "status_code": 200,
  "data": { ... },
  "metadata": {
    "timestamp": "2026-09-27T17:15:00.124Z",
    "request_id": "req-98412-a1",
    "processing_time_ms": 18.4
  }
}
```

Error Response Envelope (HTTP 4xx / 5xx)

```
{
  "success": false,
  "status_code": 409,
  "error": {
    "code": "STOCKOUT_ERROR",
    "message": "Requested quantity exceeds available unallocated stock for SKU.",
    "field": "quantity",
    "details": {
      "sku_code": "SKU-MILK-1L",
      "requested_qty": 5,
      "available_qty": 2
    }
  },
  "metadata": {
    "timestamp": "2026-09-27T17:15:00.180Z",
    "request_id": "req-98412-a2"
  }
}
```

2. API Endpoints by Domain Module**2.1 Module M-01: Authentication & Access Control****`POST /auth/login`**

Authenticates a user and issues dual JWT tokens.

- **Access:** Public
- **Request Body:**

```
{
  "username_or_email": "cashier.dhanmondi",
  "password": "Password123!"
}
```

- **Response (HTTP 200 OK):**

```
{
  "success": true,
  "data": {
    "user": {
      "user_id": 6,
      "username": "cashier.dhanmondi",
      "full_name": "Nusrat Jahan",
      "role": "OPERATOR"
    },
    "tokens": {
```

```

    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "token_type": "Bearer",
    "expires_in_seconds": 900
  }
}
}

```

(Refresh token is simultaneously dispatched in a secure `Set-Cookie: refresh\_token=...; HttpOnly; Secure; SameSite=Strict` header).

`POST /auth/refresh`

Rotates and issues a new access token using the valid HttpOnly refresh cookie.

2.2 Module M-02: Product Master & Barcode Interrogation

`GET /products/barcode/{barcode\_ean}`

Instant sub-millisecond barcode lookup for handheld scanners and POS registers.

- **Access:** All Roles ([OPERATOR](#), [CLERK](#), [SUPERVISOR](#), [PROCUREMENT](#), [STORE_MANAGER](#), [ADMIN](#))
- **Path Parameter:** [barcode_ean](#) (e.g., [8941100234123](#))
- **Response (HTTP 200 OK):**

```

{
  "success": true,
  "data": {
    "product_id": 1,
    "sku_code": "SKU-SOIL-1L",
    "barcode_ean": "8941100234123",
    "product_name": "Bashundhara Fortified Soyabean Oil 1L Bottle",
    "category": "FMCG_DRY",
    "storage_temp": "AMBIENT",
    "unit_of_measure": "BOTTLE",
    "unit_cost": 175.00,
    "total_on_hand_stock": 2150,
    "total_allocated_stock": 120,
    "available_to_promise": 2030,
    "active_batches_count": 2
  }
}

```

2.3 Module M-04: Inbound Receiving & Digital GRN

`POST /inbound/verify-scan`

Validates an incoming carton/pallet barcode against an open Purchase Order.

- **Access:** [CLERK](#), [SUPERVISOR](#), [ADMIN](#)

- **Request Body:**

```
{
  "po_id": 1,
  "barcode_ean": "8941122450012",
  "batch_lot_number": "LOT-MILK-2026-0927",
  "mfg_date": "2026-09-27",
  "expiry_date": "2026-10-07",
  "scanned_quantity": 50
}
```

- **Response (HTTP 200 OK):**

```
{
  "success": true,
  "data": {
    "verification_status": "ACCEPTED",
    "product_name": "Milk Vita Pasteurized Liquid Milk 1L Pouch",
    "remaining_shelf_life_days": 10,
    "remaining_shelf_life_percentage": 100.0,
    "po_progress": {
      "ordered_quantity": 200,
      "previously_scanned": 100,
      "currently_scanned": 50,
      "remaining_expected": 50
    }
  }
}
```

`POST /inbound/grn/complete`

Commits the final Goods Receipt Note (GRN) into the ACID database.

- **Access:** [CLERK](#), [SUPERVISOR](#), [ADMIN](#)
- **Request Body:**

```
{
  "po_id": 1,
  "supplier_invoice_ref": "INV-MV-8912",
  "notes": "Delivered on refrigerated truck Chilled temp 3.2C",
  "items": [
    {
      "product_id": 2,
      "batch_lot_number": "LOT-MILK-2026-0927",
      "location_id": 4,
      "mfg_date": "2026-09-27",
      "expiry_date": "2026-10-07",
      "accepted_quantity": 196,
      "damaged_quantity": 4,
      "damage_reason": "LEAKING_POUCH"
    }
  ]
}
```

```
}

```

- **Response (HTTP 201 Created):**

```
{
  "success": true,
  "data": {
    "grn_id": 4,
    "grn_number": "GRN-2026-0004",
    "po_number": "PO-2026-0001",
    "total_accepted": 196,
    "total_damaged": 4,
    "credit_note_advisory_bdt": 360.00,
    "status": "COMPLETED",
    "timestamp": "2026-09-27T17:22:15Z"
  }
}
```

2.4 Module M-05: Directed Spatial Putaway

`POST /locations/putaway-recommendation`

Calculates optimal bin destination for a newly received pallet based on SKU velocity and temperature constraints.

- **Access:** OPERATOR, CLERK, SUPERVISOR
- **Request Body:**

```
{
  "product_id": 2,
  "quantity": 196
}
```

- **Response (HTTP 200 OK):**

```
{
  "success": true,
  "data": {
    "recommended_location": {
      "location_id": 4,
      "zone_code": "ZONE-CHILL",
      "aisle_number": "A01",
      "rack_number": "R01",
      "shelf_tier": "S1",
      "bin_barcode": "BIN-CHL-A01-R01-S1-B01",
      "available_capacity_units": 252,
      "reason": "Chilled temperature match; Velocity Class-A closest to dispatch dock."
    }
  }
}
```

```
}

```

2.5 POS Synchronization: Real-Time Atomic Stock Deduction

POST /pos/sync

CRITICAL SLA: ≤ 2.0 seconds total round-trip ($p95 \leq 800$ ms). Executes atomic row-level locking on earliest active FEFO batch.

- **Access:** Authorized POS Registers / Cashiers
- **Headers:** **X-Idempotency-Key:** <UUIDv4>
- **Request Body:**

```
{
  "register_code": "POS-DHN-01",
  "receipt_number": "REC-DHN-2026-89142",
  "cashier_user_id": 6,
  "line_items": [
    {
      "barcode_ean": "8941122450012",
      "quantity": 2
    },
    {
      "barcode_ean": "8941100234123",
      "quantity": 1
    }
  ]
}
```

- **Response (HTTP 200 OK):**

```
{
  "success": true,
  "data": {
    "transaction_id": 14920,
    "sync_status": "CONFIRMED",
    "processed_items": [
      {
        "sku_code": "SKU-MILK-1L",
        "deducted_qty": 2,
        "batch_used": "LOT-MILK-2026-0925",
        "batch_expiry": "2026-10-01",
        "remaining_batch_qty": 46
      },
      {
        "sku_code": "SKU-SOIL-1L",
        "deducted_qty": 1,
        "batch_used": "LOT-SOIL-2026-08A",
        "batch_expiry": "2027-08-01",

```

```

    "remaining_batch_qty": 949
  }
],
"execution_time_ms": 32.1
}
}

```

`POST /pos/bulk-sync`

Bulk endpoint invoked by POS clients upon recovering from internet network outages.

- **Request Body:** An array of buffered sale transactions with client timestamps.
- **Response (HTTP 200 OK):** Itemizes committed transactions and flags any reconciliation discrepancies.

2.6 Module M-07: Replenishment & Decision Support Engine (DSS)

`GET /replenishment/calculate-eoq/{product\_id}`

Computes dynamic Economic Order Quantity and Greasley's Statistical Safety Stock using live parameters.

- **Access:** [PROCUREMENT](#), [SUPERVISOR](#), [ADMIN](#)
- **Response (HTTP 200 OK):**

```

{
  "success": true,
  "data": {
    "product_id": 1,
    "sku_code": "SKU-SOIL-1L",
    "annual_demand_D": 24000,
    "fixed_order_cost_S": 1200.00,
    "annual_holding_cost_H": 18.00,
    "calculated_eoq_units": 1789,
    "greasley_safety_stock": {
      "average_daily_demand_d": 65.75,
      "demand_std_dev_sigma_d": 12.00,
      "average_lead_time_days_L": 7.00,
      "lead_time_std_dev_sigma_L": 1.50,
      "service_level_target": "95%",
      "z_score": 1.65,
      "safety_stock_units": 174
    },
    "reorder_point_rop": 634,
    "current_inventory_position": 610,
    "replenishment_status": "REORDER_TRIGGERED"
  }
}

```

2.7 Module M-09: Cycle Counting & Anomaly Detection

`POST /audit/cycle-count/submit`

Submits blind physical count entries from warehouse floor workers.

- **Access:** OPERATOR, SUPERVISOR
- **Request Body:**

```
{
  "count_id": 1,
  "counts": [
    {
      "product_id": 1,
      "location_id": 1,
      "physical_counted_qty": 948
    }
  ]
}
```

- **Response (HTTP 200 OK):**

```
{
  "success": true,
  "data": {
    "count_id": 1,
    "items_reconciled": 1,
    "discrepancies_found": 1,
    "variance_summary": [
      {
        "sku_code": "SKU-SOIL-1L",
        "expected_qty": 950,
        "counted_qty": 948,
        "variance": -2,
        "discrepancy_value_bdt": -350.00,
        "requires_supervisor_approval": true
      }
    ]
  }
}
```

MODULE 6: SPRINT IMPLEMENTATION PLAN & TASK BREAKDOWN

Sprint Implementation Plan & Engineering Task Breakdown

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: 14-Week Agile Scrum Roadmap, Sprint Backlogs, Story Points, and Definition of Done

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Scrum Framework Overview & Velocity Model

RetailSync follows an **Agile Scrum Development Lifecycle** structured across **six 2-week sprints** and a final **2-week hardening and defense preparation phase** (14 weeks total):

- **Sprint Cadence:** 2 weeks per sprint.
- **Sprint Planning:** Day 1 of each sprint (2 hours).
- **Daily Standup:** Asynchronous or 10-minute sync.
- **Sprint Review & Demo:** Alternate Friday with academic supervisor / evaluators.
- **Definition of Done (DoD):** Code committed to Git, unit tests passing (> 80% coverage), API endpoints documented in OpenAPI, database migrations reproducible via Docker, and UI responsive on mobile viewport (375x667).

14-WEEK CAPSTONE IMPLEMENTATION ROADMAP	
Sprint 1 (W01-W02) Database Normalization, Core DDL Migrations, Auth & Next.js PWA Scaffolding	
Sprint 2 (W03-W04) Inbound PO Lifecycle, Handheld Barcode Dock Scanning, and Digital GRN Generation	
Sprint 3 (W05-W06) Directed Spatial Putaway Engine, 2D Floor Map & Real-Time FEFO Inventory Ledger	
Sprint 4 (W07-W08) Sub-2.0s POS Concurrency Deduction Engine, Row-Locking & Offline Caching Queue	
Sprint 5 (W09-W10) Replenishment DSS (EOQ, Greasley Safety Stock, ROP) & Outbound Wave Store Dispatch	
Sprint 6 (W11-W12) Blind Cycle Counting, Shrinkage Anomaly Detection & Executive Floor Telemetry	
Sprint 7 (W13-W14) Concurrency Stress-Testing (Locust 50 req/s), Security Audit, Capstone Viva Demo	

2. Sprint Backlogs & Granular Task Breakdowns

2.1 Sprint 1 (Weeks 1-2): Architecture, Relational Schema & Auth Core

- **Sprint Goal:** Establish the reproducible Docker environment, initialize the 3NF PostgreSQL database, and deliver secure JWT authentication with role-based access control.

- **Target Velocity:** 32 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-101	Setup Docker Compose orchestrating PostgreSQL 16, Redis 7, FastAPI backend, and Next.js frontend.	DevOps & Infra	5	Single command <code>docker compose up -d</code> boots all containers with healthy status.
TSK-102	Implement Alembic migration scripts for 15 core database tables and composite B-Tree indexes.	Data Modeling	8	<code>alembic upgrade head</code> runs cleanly; database schema passes 3NF verification audit.
TSK-103	Load initial authentic seed data (5 suppliers, 7 SKUs, 6 bins, 6 batches, 3 stores).	Data Modeling	3	Seed script populates DB; verify foreign key constraints prevent orphaned records.
TSK-104	Develop User entity, Argon2id password hashing, and JWT token issuance (<code>/api/v1/auth/login</code>).	Security & Auth	5	Pytest confirms valid token issued; invalid credentials return HTTP 401.
TSK-105	Implement RBAC middleware enforcing permissions for Cashier, Operator, Clerk, Supervisor, Procurement.	Security & Auth	5	Test unauthorized route access returns <code>HTTP 403 Forbidden</code> .
TSK-106	Scaffold Next.js 14 frontend design system with Tailwind CSS and responsive high-contrast layout.	Frontend PWA	6	Dashboard renders cleanly on mobile screen (375px) and desktop monitor (1920px).

2.2 Sprint 2 (Weeks 3–4): Inbound PO & Handheld Barcode Dock Receiving

- **Sprint Goal:** Deliver working dock receiving workflow allowing clerks to scan delivered carton barcodes against open POs and commit digital GRNs with damaged quarantine handling.
- **Target Velocity:** 34 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-201	Develop Supplier and Purchase Order CRUD API endpoints (<code>/api/v1/suppliers</code> , <code>/api/v1/inbound/po</code>).	Inbound Receiving	5	OpenAPI docs render PO status lifecycle transitions cleanly.

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-202	Integrate Html5-QRCode / ZXing camera and Bluetooth scanner input in Next.js PWA.	Mobile Scanning	8	Scanner decodes EAN-13 barcodes in < 350 ms with audio beep feedback.
TSK-203	Build PO line-item scan verification endpoint with remaining shelf-life check ($\geq 75\%$).	Inbound Receiving	5	Batches with < 75% shelf life are flagged with visual rejection banner.
TSK-204	Implement damaged carton logging with optional photo upload and quarantine status assignment.	Inbound Receiving	5	Quarantined units saved to product_batches with quarantine_status='QUARANTINED' .
TSK-205	Build atomic GRN completion transaction generating digital GRN and supplier credit note advisory.	Inbound Receiving	8	Commits GRN; creates inventory_transactions records; updates PO to COMPLETED .
TSK-206	Design printable internal warehouse carton/pallet barcode labels (EAN-128 format).	Inbound Receiving	3	Label layout prints cleanly on 4x6 inch thermal receipt/label printer.

2.3 Sprint 3 (Weeks 5–6): Directed Putaway & Real-Time FEFO Inventory Ledger

- **Sprint Goal:** Enable guided pallet putaway into warehouse bins and enforce automated First-Expired, First-Out (FEFO) batch priority allocation.
- **Target Velocity:** 35 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-301	Implement warehouse spatial bin layout API (locations) with zone and capacity constraints.	Spatial Putaway	5	Database rejects putaway suggestions into bins exceeding 500 kg or 300 units.
TSK-302	Develop Directed Putaway recommendation algorithm factoring SKU velocity (ABC) and temperature.	Spatial Putaway	8	Class-A chilled milk correctly routed to Chilled Zone Aisle 1 near dispatch dock.

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-303	Build mobile Directed Putaway PWA screen requiring destination bin scan confirmation.	Mobile Scanning	5	Operator cannot commit putaway without scanning the correct target bin barcode.
TSK-304	Develop Real-Time FEFO Priority Queue sorting batches by ascending expiry_date .	FEFO Engine	8	API test verifies stock allocation always selects nearest valid expiry batch.
TSK-305	Implement automated Expiry Watchlist background job (Safe, Watch, Warning, Critical ≤ 14 days).	FEFO Engine	5	Batches reaching expiration are automatically locked to quarantine_status='EXPIRED' .
TSK-306	Build interactive 2D spatial bin occupancy floor map in Next.js using CSS Grid / Canvas.	Frontend Visual	4	Bins color-code dynamically: Green (< 70%), Amber (70-90%), Red (> 90%).

2.4 Sprint 4 (Weeks 7-8): Sub-2.0s POS Sync API & Concurrency Row-Locking

- **Sprint Goal:** Build and stress-test the real-time POS stock deduction API, guaranteeing sub-2.0s response times and zero deadlocks across multiple registers, backed by offline client caching.
- **Target Velocity:** 36 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-401	Implement /api/v1/pos/sync with pessimistic row-level locking (SELECT ... FOR UPDATE).	POS Concurrency	8	Automated benchmark demonstrates database lock duration < 25 milliseconds.
TSK-402	Implement X-Idempotency-Key token validation in Redis with 48-hour TTL.	POS Concurrency	5	Replaying duplicate POST request returns cached confirmation without double-deduction.
TSK-403	Develop POS register registry and telemetry heartbeat tracking online/offline status.	POS Concurrency	5	Dashboards highlight disconnected store cash registers in real time.

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-404	Implement client-side Service Worker and encrypted IndexedDB offline sales queue.	Offline Resilience	8	Disconnecting network allows 20 scans locally; auto-syncs upon link reconnect.
TSK-405	Develop bulk synchronization endpoint (/api/v1/pos/bulk-sync) for network recovery replay.	Offline Resilience	5	Bulk sync correctly processes 50 buffered offline transactions in order.
TSK-406	Build Locust load-test script simulating 10 POS cashiers competing for the last unit of milk.	QA & Testing	5	100% of race conditions resolve cleanly; zero negative inventory balances occur.

2.5 Sprint 5 (Weeks 9–10): Replenishment DSS & Outbound Wave Dispatch

- **Sprint Goal:** Implement mathematical inventory decision support models (EOQ, Greasley Safety Stock, ROP) and multi-branch outbound store wave picking.
- **Target Velocity:** 33 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-501	Implement Dynamic EOQ formula module using historical annual demand and holding costs.	Decision Support	5	Computed EOQ matches manual mathematical formula verification within 0.01%.
TSK-502	Implement Greasley's Statistical Safety Stock formula factoring daily demand and vendor lead-time variance.	Decision Support	8	Output accounts for configured Z-scores (90%, 95%, 99%) and vendor \sigma_L.
TSK-503	Build dynamic ROP alert engine triggering automated draft Purchase Orders for procurement review.	Decision Support	5	Falling below ROP generates pre-filled draft PO ready for one-click approval.
TSK-504	Develop Branch Store Requisition Portal allowing store managers to submit weekly orders.	Store Requisition	5	Store order displays real-time central warehouse stock availability.

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-505	Build Wave Picking consolidation algorithm aggregating multiple store requisitions into one wave.	Outbound Picking	5	Aggregates duplicate SKU requests into single bulk pick task.
TSK-506	Implement shortest-path pick-list routing ordering picker travel by Aisle and Shelf sequence.	Outbound Picking	5	Picker walking distance reduced by > 40% compared to unsequenced list.

2.6 Sprint 6 (Weeks 11-12): Cycle Counting, Anomaly Detection & BI Dashboards

- **Sprint Goal:** Implement continuous blind cycle counting, unsupervised ML shrinkage anomaly detection, and executive KPI analytics.
- **Target Velocity:** 30 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-601	Implement Blind Cycle Counting sheets (system quantities hidden from floor counters).	Stocktaking & Audit	5	Mobile UI requires manual number input; reveals discrepancy only after submission.
TSK-602	Build supervisor discrepancy approval workflow with mandatory reason codes and photo proofs.	Stocktaking & Audit	5	Adjustments > 1,000 BDT require supervisor authorization before ledger commit.
TSK-603	Implement Scikit-learn IsolationForest model to detect anomalous shrinkage clusters.	Machine Learning	8	Model flags synthetic theft clusters across specific shift operators (F1 > 0.85).
TSK-604	Build Executive KPI Dashboard (Total Inventory Value, GMROI, Turnover Ratio, Spoilage Rate).	Executive BI	6	Dashboard renders historical trends using Chart.js / Recharts with sub-1.5s load.
TSK-605	Enforce database-level immutable audit ledger trigger on inventory_transactions .	Security & Audit	6	Direct SQL DELETE or UPDATE statements are blocked by database trigger exception.

2.7 Sprint 7 (Weeks 13–14): Concurrency Stress-Testing, UAT & Viva Defense

- **Sprint Goal:** Conduct comprehensive load testing (50 req/s), complete end-to-end user acceptance testing (UAT), and prepare the interactive defense presentation.
- **Target Velocity:** 25 Story Points

Task ID	Task Description	Epic	Est. Points	DoD Verification Step
TSK-701	Execute high-throughput Locust stress test (50 concurrent scanners, 10 POS registers).	Stress Testing	8	99th percentile response time remains < 1.5 seconds with zero database errors.
TSK-702	Conduct OWASP ZAP security vulnerability scan on all authentication and API endpoints.	Security Audit	5	Zero Critical or High vulnerabilities identified; SSL Labs A+ rating verified.
TSK-703	Execute complete end-to-end UAT scenario covering receiving dock to retail POS checkout.	System Testing	5	All 6 personas execute workflows seamlessly on test hardware.
TSK-704	Finalize Capstone Project Proposal Part 1 & Part 2 documentation and interactive slides.	Academic Defense	7	Deliverables compiled to PDF; interactive showcase dashboard deployed online.

APPENDIX A: COMPLETE PRODUCTION POSTGRESQL 16 DDL SCRIPT

```
-- =====
-- RetailSync Enterprise Relational Database Schema Blueprint & Seed Data (DDL)
-- Database Engine: PostgreSQL 16+ (ACID Compliant, 3NF Normalized)
-- Course: SE-231 (Software System Analysis & Design / Capstone Project 2)
-- Author: Raisul Islam Likhon (Section: SWE-44D)
-- Academic Institution: Daffodil International University (DIU)
-- Date: September 2026 | Version: 1.0.0-RELEASE
-- =====

-- -----
-- 0. DATABASE EXTENSIONS
-- -----

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- -----
-- 1. ENUMS & DOMAIN TYPES
-- -----

CREATE TYPE user_role_enum AS ENUM (
    'OPERATOR',
    'CLERK',
    'SUPERVISOR',
    'PROCUREMENT',
    'STORE_MANAGER',
    'ADMIN'
);

CREATE TYPE product_category_enum AS ENUM (
    'FMCG_DRY',
    'PERISHABLE_PRODUCE',
    'DAIRY_CHILLED',
    'FROZEN_FOODS',
    'HOUSEHOLD_NONFOOD'
);

CREATE TYPE storage_temp_enum AS ENUM (
    'AMBIENT',
    'AIR_CONDITIONED',
    'CHILLED_COLD',
    'DEEP_FROZEN'
);

CREATE TYPE batch_status_enum AS ENUM (
    'AVAILABLE',
    'QUARANTINED',
    'EXPIRED',
    'DEPLETED'
);

CREATE TYPE po_status_enum AS ENUM (
```

```

'DRAFT',
'PENDING_APPROVAL',
'ISSUED',
'PARTIAL_RECEIVED',
'COMPLETED',
'CANCELLED'
);

CREATE TYPE transaction_type_enum AS ENUM (
'INBOUND_GRN',
'DIRECTED_PUTAWAY',
'POS_SALE_FEFO',
'STORE_DISPATCH',
'DAMAGE_QUARANTINE',
'CYCLE_COUNT_ADJUSTMENT',
'SUPPLIER_RETURN'
);

CREATE TYPE requisition_status_enum AS ENUM (
'SUBMITTED',
'ALLOCATED',
'PICKING_WAVE',
'IN_TRANSIT',
'DELIVERED',
'RECONCILED'
);

-----
-- 2. TABLE DEFINITIONS (3NF NORMALIZED)
-----

-- 2.1 Users & Role-Based Access Control
CREATE TABLE users (
user_id SERIAL PRIMARY KEY,
username VARCHAR(50) UNIQUE NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
full_name VARCHAR(150) NOT NULL,
role user_role_enum NOT NULL DEFAULT 'OPERATOR',
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
last_login_at TIMESTAMP WITH TIME ZONE
);

-- 2.2 Suppliers Master
CREATE TABLE suppliers (
supplier_id SERIAL PRIMARY KEY,
company_name VARCHAR(150) NOT NULL,
trade_license_no VARCHAR(80) UNIQUE,
contact_person VARCHAR(100),
contact_email VARCHAR(100) NOT NULL,
phone_number VARCHAR(25) NOT NULL,

```

```

    avg_lead_time_days NUMERIC(5,2) NOT NULL DEFAULT 7.00,
    lead_time_std_dev NUMERIC(5,2) NOT NULL DEFAULT 1.50,
    payment_terms VARCHAR(50) DEFAULT 'Net 30',
    is_approved BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.3 Master Product Catalog
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    sku_code VARCHAR(50) UNIQUE NOT NULL,
    barcode_ean VARCHAR(30) UNIQUE NOT NULL,
    product_name VARCHAR(200) NOT NULL,
    category product_category_enum NOT NULL,
    storage_temp storage_temp_enum NOT NULL DEFAULT 'AMBIENT',
    unit_of_measure VARCHAR(20) NOT NULL DEFAULT 'UNIT',
    unit_cost NUMERIC(10,2) NOT NULL CHECK (unit_cost > 0),
    holding_cost_annual NUMERIC(10,2) NOT NULL CHECK (holding_cost_annual >= 0),
    ordering_cost_fixed NUMERIC(10,2) NOT NULL CHECK (ordering_cost_fixed >= 0),
    min_shelf_life_receiving_pct NUMERIC(5,2) NOT NULL DEFAULT 75.00,
    reorder_point INT NOT NULL DEFAULT 0,
    safety_stock_threshold INT NOT NULL DEFAULT 0,
    supplier_id INT REFERENCES suppliers(supplier_id) ON DELETE RESTRICT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.4 Spatial Warehouse Location Topology
CREATE TABLE locations (
    location_id SERIAL PRIMARY KEY,
    zone_code VARCHAR(15) NOT NULL,
    aisle_number VARCHAR(10) NOT NULL,
    rack_number VARCHAR(10) NOT NULL,
    shelf_tier VARCHAR(10) NOT NULL,
    bin_barcode VARCHAR(30) UNIQUE NOT NULL,
    capacity_units INT NOT NULL DEFAULT 300,
    capacity_kg NUMERIC(8,2) NOT NULL DEFAULT 500.00,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

-- 2.5 Perishable & Batch Inventory (FEFO/FIFO Engine)
CREATE TABLE product_batches (
    batch_id SERIAL PRIMARY KEY,
    batch_lot_number VARCHAR(60) UNIQUE NOT NULL,
    product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
    location_id INT NOT NULL REFERENCES locations(location_id) ON DELETE RESTRICT,
    mfg_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    initial_quantity INT NOT NULL CHECK (initial_quantity > 0),
    current_quantity INT NOT NULL CHECK (current_quantity >= 0),
    allocated_quantity INT NOT NULL DEFAULT 0 CHECK (allocated_quantity >= 0),
    quarantine_status batch_status_enum NOT NULL DEFAULT 'AVAILABLE',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

```

    CONSTRAINT chk_batch_quantities CHECK (allocated_quantity <= current_quantity),
    CONSTRAINT chk_mfg_before_expiry CHECK (mfg_date <= expiry_date)
);

-- 2.6 Digital Purchase Orders (PO)
CREATE TABLE purchase_orders (
    po_id SERIAL PRIMARY KEY,
    po_number VARCHAR(50) UNIQUE NOT NULL,
    supplier_id INT NOT NULL REFERENCES suppliers(supplier_id) ON DELETE RESTRICT,
    total_amount NUMERIC(12,2) NOT NULL DEFAULT 0.00,
    status po_status_enum NOT NULL DEFAULT 'DRAFT',
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    expected_delivery_date DATE NOT NULL,
    approved_by_user_id INT REFERENCES users(user_id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.7 Purchase Order Line Items
CREATE TABLE po_line_items (
    po_line_id SERIAL PRIMARY KEY,
    po_id INT NOT NULL REFERENCES purchase_orders(po_id) ON DELETE CASCADE,
    product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
    ordered_quantity INT NOT NULL CHECK (ordered_quantity > 0),
    received_quantity INT NOT NULL DEFAULT 0 CHECK (received_quantity >= 0),
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),
    CONSTRAINT uq_po_product UNIQUE (po_id, product_id)
);

-- 2.8 Inbound Goods Receipt Notes (GRN)
CREATE TABLE goods_receipt_notes (
    grn_id SERIAL PRIMARY KEY,
    grn_number VARCHAR(50) UNIQUE NOT NULL,
    po_id INT NOT NULL REFERENCES purchase_orders(po_id) ON DELETE RESTRICT,
    supplier_invoice_ref VARCHAR(80),
    accepted_quantity INT NOT NULL DEFAULT 0,
    rejected_damaged_quantity INT NOT NULL DEFAULT 0,
    receiving_clerk_id INT NOT NULL REFERENCES users(user_id),
    notes TEXT,
    received_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.9 Immutable Inventory Transactions Audit Ledger (ACID Row-Locking)
CREATE TABLE inventory_transactions (
    transaction_id BIGSERIAL PRIMARY KEY,
    product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
    batch_id INT REFERENCES product_batches(batch_id) ON DELETE RESTRICT,
    location_id INT NOT NULL REFERENCES locations(location_id) ON DELETE RESTRICT,
    transaction_type transaction_type_enum NOT NULL,
    quantity INT NOT NULL CHECK (quantity <> 0),
    reference_document_no VARCHAR(80),
    user_id INT NOT NULL REFERENCES users(user_id),
    recorded_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP

```

```

);

-- 2.10 Retail Branch Stores Directory
CREATE TABLE branch_stores (
  store_id SERIAL PRIMARY KEY,
  store_code VARCHAR(30) UNIQUE NOT NULL,
  store_name VARCHAR(150) NOT NULL,
  location_area VARCHAR(100) NOT NULL,
  store_manager_user_id INT REFERENCES users(user_id),
  contact_phone VARCHAR(25) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.11 Retail Branch POS Cash Registers & Sync Telemetry
CREATE TABLE pos_registers (
  register_id SERIAL PRIMARY KEY,
  register_code VARCHAR(30) UNIQUE NOT NULL,
  store_id INT NOT NULL REFERENCES branch_stores(store_id) ON DELETE RESTRICT,
  terminal_ip_mac VARCHAR(60) NOT NULL,
  is_online BOOLEAN NOT NULL DEFAULT TRUE,
  last_sync_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.12 Store Stock Requisitions
CREATE TABLE store_requisitions (
  requisition_id SERIAL PRIMARY KEY,
  requisition_no VARCHAR(50) UNIQUE NOT NULL,
  store_id INT NOT NULL REFERENCES branch_stores(store_id) ON DELETE RESTRICT,
  status requisition_status_enum NOT NULL DEFAULT 'SUBMITTED',
  required_by_date DATE NOT NULL,
  dispatch_vehicle_plate VARCHAR(30),
  submitted_by_user_id INT NOT NULL REFERENCES users(user_id),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.13 Store Requisition Line Items
CREATE TABLE requisition_line_items (
  req_line_id SERIAL PRIMARY KEY,
  requisition_id INT NOT NULL REFERENCES store_requisitions(requisition_id) ON DELETE CASCADE,
  product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
  requested_qty INT NOT NULL CHECK (requested_qty > 0),
  allocated_qty INT NOT NULL DEFAULT 0,
  dispatched_qty INT NOT NULL DEFAULT 0,
  received_qty INT NOT NULL DEFAULT 0,
  CONSTRAINT uq_req_product UNIQUE (requisition_id, product_id)
);

-- 2.14 Physical Stocktaking Cycle Counts
CREATE TABLE cycle_counts (
  count_id SERIAL PRIMARY KEY,
  count_code VARCHAR(50) UNIQUE NOT NULL,

```

```

zone_code VARCHAR(15) NOT NULL,
scheduled_date DATE NOT NULL DEFAULT CURRENT_DATE,
status VARCHAR(25) NOT NULL DEFAULT 'IN_PROGRESS' CHECK (status IN ('IN_PROGRESS',
'PENDING_APPROVAL', 'COMPLETED')),
conducted_by_user_id INT NOT NULL REFERENCES users(user_id),
approved_by_user_id INT REFERENCES users(user_id),
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2.15 Cycle Count Items (Blind Physical Verification)
CREATE TABLE cycle_count_items (
count_item_id SERIAL PRIMARY KEY,
count_id INT NOT NULL REFERENCES cycle_counts(count_id) ON DELETE CASCADE,
product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
location_id INT NOT NULL REFERENCES locations(location_id) ON DELETE RESTRICT,
system_expected_qty INT NOT NULL, -- Masked from floor counter UI
physical_counted_qty INT, -- Entered blindly by counter
variance_qty INT GENERATED ALWAYS AS (physical_counted_qty - system_expected_qty) STORED,
discrepancy_reason VARCHAR(100),
counted_at TIMESTAMP WITH TIME ZONE
);

-----
-- 3. STRATEGIC B-TREE INDEXES FOR SUB-SECOND PERFORMANCE
-----

CREATE INDEX idx_products_barcode ON products(barcode_ean);
CREATE INDEX idx_products_sku ON products(sku_code);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_batches_fefo_queue ON product_batches(product_id, expiry_date ASC, quarantine_status)
WHERE quarantine_status = 'AVAILABLE';
CREATE INDEX idx_batches_expiry ON product_batches(expiry_date);
CREATE INDEX idx_transactions_product_date ON inventory_transactions(product_id, recorded_at DESC);
CREATE INDEX idx_transactions_batch_id ON inventory_transactions(batch_id);
CREATE INDEX idx_locations_bin_barcode ON locations(bin_barcode);
CREATE INDEX idx_po_number ON purchase_orders(po_number);
CREATE INDEX idx_pos_registers_store ON pos_registers(store_id);

-----
-- 4. DATABASE INTEGRITY TRIGGERS
-----

-- 4.1 Enforce Immutable Append-Only Ledger (Block UPDATE & DELETE)
CREATE OR REPLACE FUNCTION prevent_transaction_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Security Violation: Records in inventory_transactions are immutable and cannot be
updated or deleted.';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_immutable_inventory_transactions
BEFORE UPDATE OR DELETE ON inventory_transactions

```

FOR EACH ROW

EXECUTE FUNCTION prevent_transaction_modification();

-- 5. SEED DATA (AUTHENTIC BANGLADESHI SUPER SHOP DOMAIN)

-- 5.1 System Users (Default Password: "Password123!" hashed with bcrypt)

INSERT INTO users (username, email, password_hash, full_name, role) VALUES

('likhon.admin', 'likhon@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Raisul Islam Likhon', 'ADMIN'),
('tariqul.floor', 'tariqul@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Tariqul Islam', 'SUPERVISOR'),
('salma.procure', 'salma@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Salma Akter', 'PROCUREMENT'),
('clerk.dock', 'clerk@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Rafiqul Hasan', 'CLERK'),
('operator.kamal', 'kamal@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Kamal Hossain', 'OPERATOR'),
('cashier.dhanmondi', 'cashier1@retailsync.com',
'\$2b\$12\$eX8m3Ym1d6zWz9tK2jK8OeZfQ3U8L2yE5q5K8Z7wQ3U8L2yE5q5K8', 'Nusrat Jahan', 'OPERATOR');

-- 5.2 Suppliers (Major Bangladeshi FMCG & Dairy Brands)

INSERT INTO suppliers (company_name, trade_license_no, contact_person, contact_email, phone_number, avg_lead_time_days, lead_time_std_dev, payment_terms) VALUES

('Bashundhara Food & Beverage Ltd.', 'TL-DHK-2018-9841', 'Farhan Chowdhury', 'farhan@bashundhara.com',
'+8801711223344', 7.00, 1.50, 'Net 30'),
('Milk Vita (Bangladesh Milk Producers Co-Operative)', 'TL-DHK-2015-1120', 'Aminul Islam',
'orders@milkvita.org.bd', '+8801819334455', 2.00, 0.50, 'Net 15'),
('Square Consumer Products Ltd. (Radhuni/Chashi)', 'TL-DHK-2017-4521', 'Tanvir Ahmed',
'supply@squaregroup.com', '+8801912445566', 5.00, 1.20, 'Net 30'),
('PRAN Agro Business Ltd.', 'TL-DHK-2016-8742', 'Mahmudur Rahman', 'dispatch@prangroup.com',
'+8801613556677', 6.00, 1.80, 'Net 30'),
('Akij Food & Beverage Ltd.', 'TL-DHK-2019-3321', 'Saiful Karim', 'orders@akij.net', '+8801514667788', 4.00, 1.00,
'Net 15');

-- 5.3 Master Products Catalog

INSERT INTO products (sku_code, barcode_ean, product_name, category, storage_temp, unit_of_measure, unit_cost, holding_cost_annual, ordering_cost_fixed, min_shelf_life_receiving_pct, reorder_point, safety_stock_threshold, supplier_id) VALUES

('SKU-SOIL-1L', '8941100234123', 'Bashundhara Fortified Soyabean Oil 1L Bottle', 'FMCG_DRY', 'AMBIENT',
'BOTTLE', 175.00, 18.00, 1200.00, 75.00, 1218, 320, 1),
('SKU-MILK-1L', '8941122450012', 'Milk Vita Pasteurized Liquid Milk 1L Pouch', 'DAIRY_CHILLED',
'CHILLED_COLD', 'POUCH', 90.00, 22.00, 500.00, 80.00, 250, 85, 2),
('SKU-RICE-5K', '8941144781290', 'Chashi Aromatic Chinigura Rice 5kg Poly', 'FMCG_DRY', 'AMBIENT', 'BAG',
720.00, 35.00, 1500.00, 70.00, 180, 45, 3),
('SKU-SUGR-1K', '8941100341908', 'Fresh Refined White Sugar 1kg Packet', 'FMCG_DRY', 'AMBIENT', 'PACKET',
135.00, 12.00, 800.00, 85.00, 400, 110, 1),
('SKU-TURM-200G', '8941144120934', 'Radhuni Pure Turmeric Powder 200g Foil', 'FMCG_DRY', 'AMBIENT', 'FOIL',
95.00, 8.00, 450.00, 80.00, 150, 35, 3),
('SKU-YGRT-500G', '8941122901234', 'Milk Vita Sweetened Dahi (Yogurt) 500g Cup', 'DAIRY_CHILLED',
'CHILLED_COLD', 'CUP', 120.00, 28.00, 600.00, 80.00, 120, 40, 2),

```
('SKU-JUIC-1L', '8941155981245', 'PRAN Frooto Mango Juice 1L Tetra Pak', 'FMCG_DRY', 'AMBIENT', 'PACK',
110.00, 15.00, 700.00, 75.00, 220, 60, 4);
```

```
-- 5.4 Spatial Warehouse Locations (Tejgaon Central Distribution Center)
```

```
INSERT INTO locations (zone_code, aisle_number, rack_number, shelf_tier, bin_barcode, capacity_units,
capacity_kg) VALUES
```

```
('ZONE-DRY', 'A01', 'R01', 'S1', 'BIN-DRY-A01-R01-S1-B01', 500, 800.00),
('ZONE-DRY', 'A01', 'R01', 'S2', 'BIN-DRY-A01-R01-S2-B02', 500, 800.00),
('ZONE-DRY', 'A02', 'R01', 'S1', 'BIN-DRY-A02-R01-S1-B01', 400, 600.00),
('ZONE-CHILL', 'A01', 'R01', 'S1', 'BIN-CHL-A01-R01-S1-B01', 300, 400.00),
('ZONE-CHILL', 'A01', 'R01', 'S2', 'BIN-CHL-A01-R01-S2-B02', 300, 400.00),
('ZONE-BULK', 'B01', 'R01', 'S1', 'BIN-BLK-B01-R01-S1-P01', 1000, 2000.00);
```

```
-- 5.5 Product Batches (FEFO Test Batches: Some Expiring Soon, Some Fresh)
```

```
INSERT INTO product_batches (batch_lot_number, product_id, location_id, mfg_date, expiry_date,
initial_quantity, current_quantity, allocated_quantity, quarantine_status) VALUES
```

```
-- Soyabean Oil (Stable FMCG)
```

```
('LOT-SOIL-2026-08A', 1, 1, '2026-08-01', '2027-08-01', 1000, 950, 0, 'AVAILABLE'),
('LOT-SOIL-2026-09B', 1, 2, '2026-09-01', '2027-09-01', 1200, 1200, 0, 'AVAILABLE'),
```

```
-- Milk Vita (High Perishable - FEFO Test: Batch A expires in 4 days, Batch B in 10 days)
```

```
('LOT-MILK-2026-0925', 2, 4, '2026-09-25', '2026-10-01', 200, 48, 0, 'AVAILABLE'),
('LOT-MILK-2026-0927', 2, 5, '2026-09-27', '2026-10-07', 300, 300, 0, 'AVAILABLE'),
```

```
-- Sweet Dahi (Perishable Dairy)
```

```
('LOT-YGRT-2026-0920', 6, 4, '2026-09-20', '2026-10-05', 150, 35, 0, 'AVAILABLE'),
('LOT-YGRT-2026-0926', 6, 5, '2026-09-26', '2026-10-18', 200, 200, 0, 'AVAILABLE');
```

```
-- 5.6 Branch Stores
```

```
INSERT INTO branch_stores (store_code, store_name, location_area, contact_phone) VALUES
```

```
('STORE-DHK-DHN', 'Shwapno Dhanmondi Flagship Outlet', 'Road 27, Dhanmondi, Dhaka', '+8801711998877'),
('STORE-DHK-UTT', 'Shwapno Uttara Sector 3 Branch', 'Sector 3, Uttara, Dhaka', '+8801811887766'),
('STORE-DHK-GUL', 'Shwapno Gulshan 2 Express Outlet', 'Gulshan Avenue, Dhaka', '+8801911776655');
```

```
-- 5.7 POS Cash Registers
```

```
INSERT INTO pos_registers (register_code, store_id, terminal_ip_mac, is_online) VALUES
```

```
('POS-DHN-01', 1, '192.168.10.101 / 00:1A:2B:3C:4D:5E', TRUE),
('POS-DHN-02', 1, '192.168.10.102 / 00:1A:2B:3C:4D:5F', TRUE),
('POS-UTT-01', 2, '192.168.20.101 / 00:1A:2B:3C:4D:6A', TRUE),
('POS-GUL-01', 3, '192.168.30.101 / 00:1A:2B:3C:4D:7B', TRUE);
```

```
-- 5.8 Initial Sample Inbound Transactions
```

```
INSERT INTO inventory_transactions (product_id, batch_id, location_id, transaction_type, quantity,
reference_document_no, user_id) VALUES
```

```
(1, 1, 1, 'INBOUND_GRN', 1000, 'GRN-2026-0001', 4),
(2, 3, 4, 'INBOUND_GRN', 200, 'GRN-2026-0002', 4),
(6, 5, 4, 'INBOUND_GRN', 150, 'GRN-2026-0003', 4);
```

```
-- End of DDL & Seed Script
```